

Aurix

Microcontroller Infineon Software Architecture

User's Manual

FEE Driver

Release v2.9

2016-08

Edition 2016-08

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2016 Infineon Technologies AG

All Rights Reserved.

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Edition 2016-08

Published by

Infineon Technologies AG

81726 Munich, Germany

© 2016 Infineon Technologies AG

All Rights Reserved.

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Document Change History		
Date	Version	Change Description
2016-08-02	2.9	Updated sections 4.5.13 to add FeeMaxBytesPerCycle parameter.
2016-06-06	2.8	Updated sections 6.2 as part of 23x release
2015-10-30	2.7	Updated as per JIRA 0000051113-19.
2015-07-02	2.6	Updated Section 4.5.3 for JIRA 0000051129-11. Default is now made FALSE.
2015-03-10	2.5	Updated Section 4 for JIRA 0000051190-23. Added note in Section 11.2.
2015-03-03	2.4	Updated for TC21x
2014-12-05	2.3	Updated as per review comments REV_006581
2014-12-02	2.2	Updated for UTP AI00251090 - I/O Mode Information
2014-08-25	2.1	Updated as per review comments of REV_006090
2014-07-24	2.0	Updated for UTP AI00236777 and AI00239664
2014-05-30	1.9	Updated description for FeeThresholdValue
2014-02-24	1.8	Updated for Aurix EP devices
2013-09-24	1.7	Updated for UTPs AI00219555 and AI00194692
2013-08-28	1.6	Updated as per review comments REV_004750
2013-08-22	1.5	Updated file structure. Added functional limitations. Updated for TC29x support.
2013-06-27	1.4	Updated for TC26x support
2013-03-25	1.3	Updated for TC27x B-Step changes and FLS VendorApilnfix changes.
2013-01-17	1.2	Updated description for FeeTotalUnconfigBlockCountMax and jump table implementation
2013-01-17	1.1	Updated as per review comments
2012-12-31	1.0	Updated for AS321
2012-07-03	0.1	Initial Version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
 Your feedback will help us to continuously improve the quality of this document.
 Please send your proposal (including a reference to this document) to:



mcdocu.comments@infineon.com

Table of Contents	Page
1 Introduction	14
1.1 Purpose of the document.....	14
1.2 Scope	14
1.3 Abbreviations and Acronyms	14
1.4 References.....	16
2 FEE Driver Overview	17
2.1 Software Hardware Mapping.....	17
2.2 Linking Different User Configurations during Run Time	18
2.3 FLS Driver Support for FEE Driver.....	18
2.4 Software Module Description	18
3 File Structure	20
4 Configuration Documentation.....	22
4.1 Configuration Concept	22
4.1.1 Configuration Class	22
4.1.2 Configuration Variant.....	22
4.1.3 How to read the Configuration Class field	23
4.2 Configuration Parameter Variance	23
4.3 Container: FeeGeneral	23
4.3.1 FeeDevErrorDetect	24
4.3.2 FeeIndex.....	24
4.3.3 FeeNvmJobEndNotification	24
4.3.4 FeeNvmJobErrorNotification	25
4.3.5 FeePollingMode	25
4.3.6 FeeSetModeSupported	26

Table of Contents		Page
4.3.7	FeeVersionInfoApi.....	26
4.3.8	FeeVirtualPageSize.....	27
4.4	Container: FeeBlockConfiguration	27
4.4.1	FeeBlockNumber.....	28
4.4.2	FeeBlockSize.....	28
4.4.3	FeeImmediateData.....	29
4.4.4	FeeNumberOfWriteCycles	29
4.4.5	FeeDeviceIndex	30
4.5	Container: FeeIcxSpecificConfig	30
4.5.1	FeeThresholdValue	30
4.5.2	FeeMaxBlockCount	32
4.5.3	FeeUseEraseSuspend	33
4.5.4	FeeStateVarStructure.....	34
4.5.5	FeeUnConfigBlock	34
4.5.6	FeeUnConfigBlkOverflowHandle	35
4.5.7	FeeGcRestart.....	35
4.5.8	FeeGetCycleCountApi	36
4.5.9	FeeEraseAllEnable	36
4.5.10	FeeGetPrevDataApi.....	37
4.5.11	FeeIllegalStateNotification.....	37
4.5.12	FeeVirginFlashIllegalState	37
4.5.13	FeeMaxBytesPerCycle	38
4.6	Container: FeePublishedInformation	39
4.6.1	FeeBlockOverhead	39
4.6.2	FeeMaximumBlockingTime.....	39

Table of Contents	Page
4.6.3 FeePageOverhead	40
4.7 FeeDemConfig	40
4.7.1 FeeGcInitDemReport	40
4.7.2 FeeWriteDemReport	41
4.7.3 FeeReadDemReport	41
4.7.4 FeeGCReadDemReport	42
4.7.5 FeeGCWriteDemReport	42
4.7.6 FeeGCeraseDemReport	43
4.7.7 FeeInvalidateDemReport	43
4.7.8 FeeWriteCyclesDemReport	44
4.7.9 FeeGcTrigDemReport	44
4.7.10 FeeUnconfigBlkExceededDemReport	45
5 Published Parameters	46
5.1 Vendor ID	46
5.2 Module ID	46
5.3 AR Major Version	46
5.4 AR Minor Version	46
5.5 AR Revision Version	47
5.6 SW Major Version	47
5.7 SW Minor Version	47
5.8 SW Patch Version	47
6 API Documentation	49
6.1 API Type Definitions	49
6.1.1 Type Definition Fee_ConfigType	49

Table of Contents		Page
6.2	API Functions	49
6.2.1	Fee_Init	49
6.2.2	Fee_SetMode	50
6.2.3	Fee_Read.....	51
6.2.4	Fee_Write.....	52
6.2.5	Fee_Cancel	53
6.2.6	Fee_GetStatus.....	54
6.2.7	Fee_GetJobResult	55
6.2.8	Fee_InvalidateBlock	57
6.2.9	Fee_GetVersionInfo	58
6.2.10	Fee_EraseImmediateBlock.....	58
6.2.11	Fee_JobEndNotification	59
6.2.12	Fee_JobErrorNotification	60
6.2.13	Fee_17_GetCycleCount.....	60
6.2.14	Fee_17_DisableGcStart	62
6.2.15	Fee_17_EnableGcStart.....	62
6.2.16	Fee_MainFunction	63
6.2.17	Fee_17_GetPrevData	64
6.2.18	Fee_17_IllegalStateNotification.....	65
6.3	Fee Feature Description	66
6.3.1	Emulation Algorithm	66
6.3.1.1	Support for Unconfigured blocks.....	67
6.3.2	Robustness Features	68
6.3.3	GC Start Disable/Enable Feature	68
6.3.3.1	Disable GC behaviour incase of Immediate Write requests.....	68

Table of Contents	Page
6.3.3.2 Disable GC behaviour incase of Normal Write / Invalidate requests.....	68
6.3.4 Behaviour of Write Requests	69
6.3.5 Behaviour incase Write/Erase failure during GC	69
6.3.6 Illegal State Notification.....	70
6.3.7 Interruption of GC.....	71
6.3.8 Interruption of Write request	71
6.3.9 Behaviour incase Write/Read failures	71
6.3.10 FEE behavior for GC_RESTART_INIT and GC_RESTART_WRITE configurations.....	72
7 Data Description	74
8 Error Classification.....	78
9 Example Usage	80
10 Configuring FEE Driver	81
10.1 Using FEE services	81
10.1.1 Function call example	81
10.1.2 Jump table functionality implementation hint	82
10.1.3 Important Notes about the usage of FEE driver	83
11 Assumptions and Limitations	84
11.1 Assumptions and Deviations from the Software Specification.....	84
11.2 Limitations.....	84
11.3 Deviations.....	85
11.4 Hardware features that are not supported	86

List of Figures	Page
Figure 1 SW HW Mapping	17
Figure 2 Interfaces of FEE driver	19
Figure 3 File structure of FEE driver.....	20
Figure 4 Fee Execution in FEE_GC_RESTART_INIT	73
Figure 5 Fee Execution in FEE_GC_RESTART_WRITE	74

List of Tables	Page
Table 1 Variance of peripheral units/features.....	14
Table 2 The abbreviations and Acronyms used inside this document	14
Table 1 File Structure Contents	20
Table 3 Configuration Class Example 1.....	23
Table 4 Configuration Class Example 2.....	23
Table 5 FeeDevErrorDetect.....	24
Table 6 FeeIndex.....	24
Table 7 FeeNvmJobEndNotification	24
Table 8 FeeNvmJobErrorNotification	25
Table 9 FeePollingMode	25
Table 10 FeeSetModeSupported	26
Table 11 FeeVersionInfoApi.....	26
Table 12 FeeVirtualPageSize.....	27
Table 13 FeeBlockNumber.....	28
Table 14 FeeBlockSize	28
Table 15 FeeImmediateData.....	29
Table 16 FeeNumberOfWriteCycles	29
Table 17 FeeDeviceIndex	30
Table 18 Specification of FeeThresholdValue.....	30
Table 19 Specification of FeeMaxBlockCount	32
Table 20 Specification of FeeUseEraseSuspend	33
Table 21 Specification of FeeStateVarStructure.....	34
Table 22 Specification of FeeUnConfigBlock	34
Table 23 Specification of FeeUnConfigBlkOverflowHandle.....	35
Table 24 Specification of FeeGcRestart	35
Table 25 Specification of FeeGetCycleCountApi	36
Table 26 Specification of FeeEraseAllEnable	36
Table 27 Specification of FeeGetPrevDataApi.....	37
Table 28 Specification of FeeIllegalStateNotification	37
Table 29 Specification of FeeVirginFlashIllegalState	37
Table 30 Specification of FeeMaxBytesPerCycle	38
Table 31 FeeBlockOverhead	39
Table 32 FeeMaximumBlockingTime.....	39
Table 33 FeePageOverhead	40
Table 34 Specification of FeeGcInitDemReport.....	40
Table 35 Specification of FeeWriteDemReport	41
Table 36 Specification of FeeReadDemReport	41
Table 37 Specification of FeeGCReadDemReport	42
Table 38 Specification of FeeGCWriteDemReport	42

List of Tables	Page
Table 39 Specification of FeeGCeraseDemReport	43
Table 40 Specification of FeeInvalidateDemReport	43
Table 41 Specification of FeeWriteCyclesDemReport	44
Table 42 Specification of FeeGcTrigDemReport	44
Table 43 Specification of FeeUnconfigBlkExceededDemReport	45
Table 44 Vendor ID	46
Table 45 Module ID	46
Table 46 FEE Driver AUTOSAR Requirement Major Version	46
Table 47 FEE Driver AUTOSAR Requirement Minor Version	46
Table 48 FEE Driver AUTOSAR Requirement Revision Version	47
Table 49 SW Major Version	47
Table 50 SW Minor Version	47
Table 51 SW Patch Version	47
Table 52 Type Definition Fee_ConfigType	49
Table 53 Specification of Fee_Init	49
Table 54 Specification of Fee_SetMode	50
Table 55 Specification of Fee_Read	51
Table 56 Specification of Fee_Write	52
Table 57 Specification of Fee_Cancel	53
Table 58 Specification of Fee_GetStatus	54
Table 59 Specification of Fee_GetJobResult	55
Table 60 Specification of Fee_InvalidateBlock	57
Table 61 Specification of Fee_GetVersionInfo	58
Table 62 Specification of Fee_EraseImmediateBlock	58
Table 63 Specification of Fee_JobEndNotification	59
Table 64 Specification of Fee_JobErrorNotification	60
Table 65 Specification of Fee_17_GetCycleCount	60
Table 66 Specification of Fee_17_DisableGcStart	62
Table 67 Specification of Fee_17_EnableGcStart	62
Table 68 Specification of Fee_MainFunction	63
Table 69 Specification of Fee_17_GetPrevData	64
Table 70 Specification of Fee_17_IllegalStateNotification	65
Table 71 Behaviour of Write Requests	69
Table 72 Behaviour in various failed GC states	70
Table 73 Global Data Description	74
Table 74 Error Classification	78

List of Code Listings

Page

Code Listing 1	Function call example	81
----------------	-----------------------------	----

1 Introduction

1.1 Purpose of the document

This user manual provides information regarding the functionality, driver configuration and API specification of the FEE Driver. This User Manual is intended to help the user to get acquainted with the FEE Driver implementation for the Aurix Hardware Platform. It is the documentation that describes how to use the FEE Driver.

1.2 Scope

This document addresses the following features of the FEE Driver implementation,

- Aurix microcontroller HW modules used to realize the Driver
- File structure of the Driver
- Configuration parameters of the Driver
- Parameters published by the Driver
- API Specification
- Common use cases with code snippets.
- Limitations and Assumptions

This documentation applies to the Fee Driver Variant: Variant PC

This document addresses the driver implementation for the following variants of the Aurix Family.

The following table gives the available units/features in the variants of Aurix Family micro-controllers for driver usage.

Table 1 Variance of peripheral units/features

Device	DF_EEPROM Size (in kB)	FEE Sector0 Size (in kB)	FEE Sector1 Size (in kB)	WL Size (in Bytes)	Page Size (in Bytes)
TC27x	384	192	192	512	8
TC26x	96	48	48	512	8
TC29x	768	384	384	512	8
TC23x	128	64	64	512	8
TC22x	96	48	48	512	8
TC24x	48	24	24	512	8
TC21x	64	32	32	512	8

1.3 Abbreviations and Acronyms

Table 2 The abbreviations and Acronyms used inside this document

Abbreviation	Explanation
--------------	-------------

MemIf	Memory Abstraction Interface
NvM	NVRAM Manager
NVRAM	Non-volatile RAM (Random Access Memory)
NVRAM block	Management unit as seen by the NVRAM Manager
(Logical) block	Smallest writable / erasable unit as seen by the modules user. Consists of one or more virtual pages.
Virtual page	May consist of one or several physical pages to ease handling of logical blocks and address calculation.
Internal residue	Unused space at the end of the last virtual page if the configured block size isn't an integer multiple of the virtual page size.
Virtual address	Consisting of 16 bit block number and 16 bit offset inside the logical block.
Physical address	Address information in device specific format that is used to access a logical block.
Dataset	Concept of the NVRAM manager: A user addressable array of blocks of the same size. E.g. could be used to provide different configuration settings for the CAN driver (CAN IDs, filter settings, ...) to an ECU which has otherwise identical application software (e.g. door module).
Redundant copy	Concept of the NVRAM manager: Storing the same information twice to enhance reliability of data storage.
FEE	Flash EEPROM Emulation
FLS	Flash
EEPROM	Electrically Erasable and Programmable ROM (Read Only Memory)
LSB	Least significant bit / byte (depending on context). Here it is bit.
MSB	Most significant bit / byte (depending on context). Here it is bit.
GC	Garbage collection
BL	Bit-Line
WL	Word-Line
WL/BL	Word-Line/Bit-Line
KB	Kilo Bytes
B	Bytes
DF_EEPROM	Data Flash dedicated for EEPROM emulation
ms	milli-seconds
INSM	Init Normal State Machine
IDSM	Init Dirty State Machine

FEE Sector	The DF_EEPROM area is logically divided into two parts referred to as “FEE sectors” for implementing the “double sector algorithm”. In this document “sector” refers to “FEE sector”, unless specified explicitly as “physical sector”.
Unconfigured block	Data block which is stored in the data flash (DF_EEPROM) but is not contained in the currently active configuration

1.4 References

This section lists down all relevant documents for this User Manual.

Some of the artifacts listed below are given without the version number. For those artifacts, refer the latest version.

[1] Specification of FEE

AUTOSAR_SWS_Flash_EEPROM_Emulation.pdf, V1.3.0

[2] Target User Manuals:

tc27xC_um_v2.0OM.pdf

tc29xB_um_v1.3.pdf

tc26xB_um_v1.3.pdf

tc23x_tc22x_um_v1.1.pdf

[3] FLS driver user manual: Aurix_MC-ISAR_UM_FLSDriver.pdf

2 FEE Driver Overview

The FEE module is part of the “Memory Hardware Abstraction” layer. The FEE shall abstract from the device specific addressing scheme and segmentation and provide the upper layers with a virtual addressing scheme and segmentation as well as a “virtually” unlimited number of erase / write cycles.

The upper layers of software should not depend on the type of the persistent data storage, i.e. if data is stored in internal Flash or internal EEPROM or external memory devices. The upper layer software should not need to care about the physical constraints of the storage device. All these device specifics should be encapsulated by the Flash EEPROM emulation.

2.1 Software Hardware Mapping

The FEE driver implements an emulated EEPROM with the data Flash (DF_EEPROM) of the Aurix family of microcontrollers. It doesn't access the DFLASH directly but via the AUTOSAR Flash driver. The complete DF_EEPROM is under the control of FEE/FLS driver. No other software component is allowed to access DF_EEPROM.

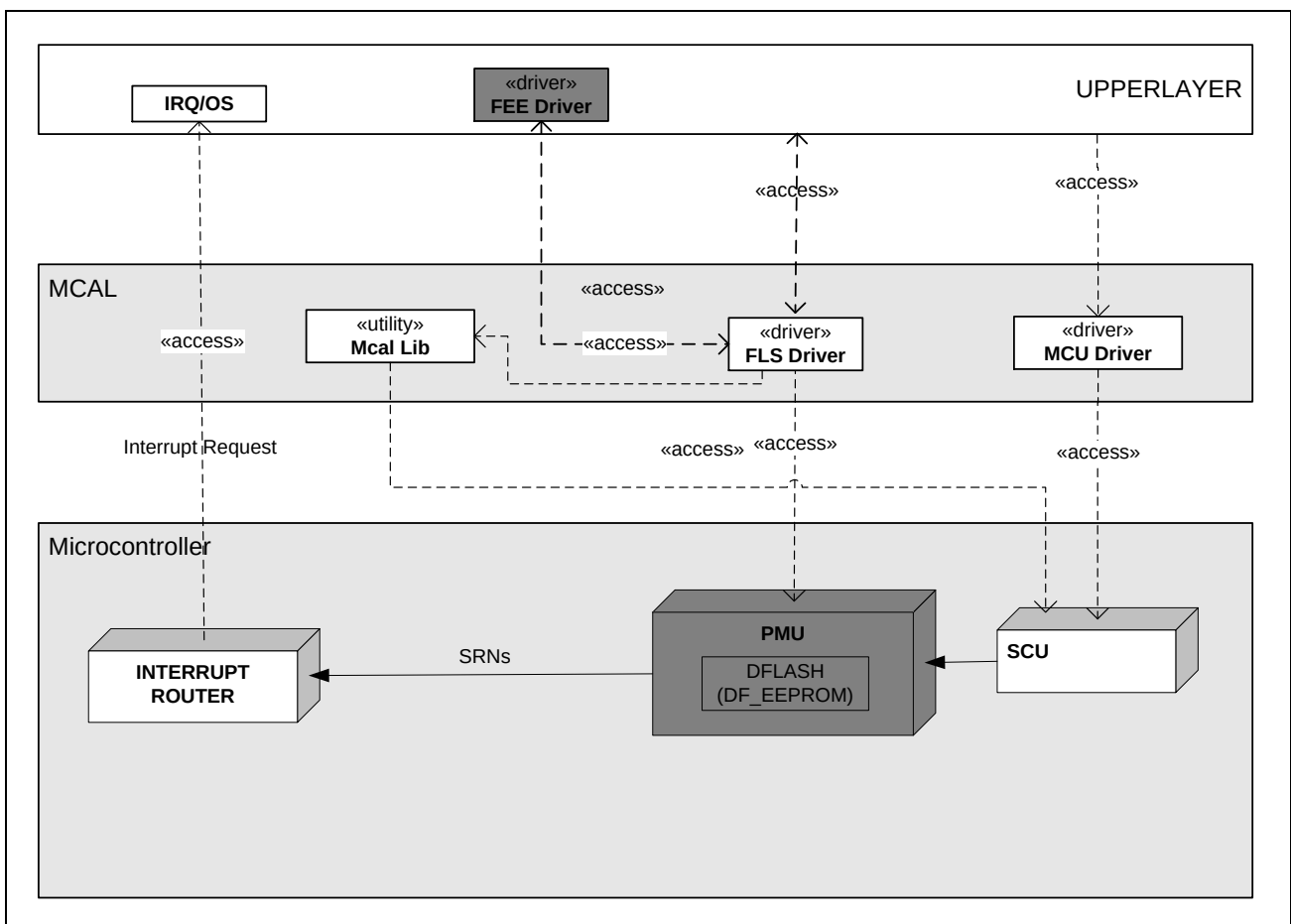


Figure 1 SW HW Mapping

2.2 Linking Different User Configurations during Run Time

This FEE driver provides user a flexibility to use a single footprint of FEE and also dynamically use a different set of user configurations without affecting their last states. For eg: After using FEE with configuration 'A', user may re-initialize FEE with a different set of configuration 'B' with following benefits.

1. While configuration 'B' is in use, the FEE state for configuration 'A' will remain intact and vice versa.
2. Linking and locating will not be a problem as the state variables are accessed via table of pointers.

2.3 FLS Driver Support for FEE Driver

FEE driver uses the FLS driver APIs Fls_17_Pmu_Read, Fls_17_Pmu_Write and Fls_17_Pmu_Erase for performing read from DF_EEPROM, write to DF_EEPROM and erase FEE Sector respectively.

FEE driver uses the FLS driver API Fls_17_Pmu_SetMode to set the mode of the underlying FLS driver to FAST/SLOW.

FEE driver uses the FLS driver API Fls_17_Pmu_ReadWordsSync to read a chunk of data synchronously.

FEE driver uses the FLS driver API Fls_17_Pmu_CompareWordsSync to compare the written data with the source data buffer synchronously.

FEE driver uses the FLS driver API Fls_17_Pmu_VerifyErase for verifying the FEE sector erase operation.

FEE driver uses the FLS driver API Fls_17_Pmu_CancelNonEraseJobs to cancel an ongoing user requested read job.

FEE driver uses the FLS driver API Fls_17_Pmu_GetNotifCaller to get the job type, which caused the FEE notification function Fee_JobEndNotification/Fee_JobErrorNotification.

As the entire DF_EEPROM area belongs to the same physical sector, both FEE sector 0 and FEE sector 1 belongs to the same physical sector. In order to carry out read or write operation on one of the FEE sector while the other FEE sector is being erased, first the ongoing erase operation has to be suspended and then the read/write operation has to be carried out and then the erase operation has to be resumed. FLS driver provides explicit APIs Fls_17_Pmu_SuspendErase and Fls_17_Pmu_ResumeErase to perform suspend-erase and resume-erase operations respectively.

Interrupts are not needed for the FEE Driver. However, if interrupts are configured for the underlying FLS driver, then the callback functions of the FEE driver (Fee_JobEndNotification, Fee_JobErrorNotification) are called by the FLS driver's ISR, when the job is finished by the FLS driver.

2.4 Software Module Description

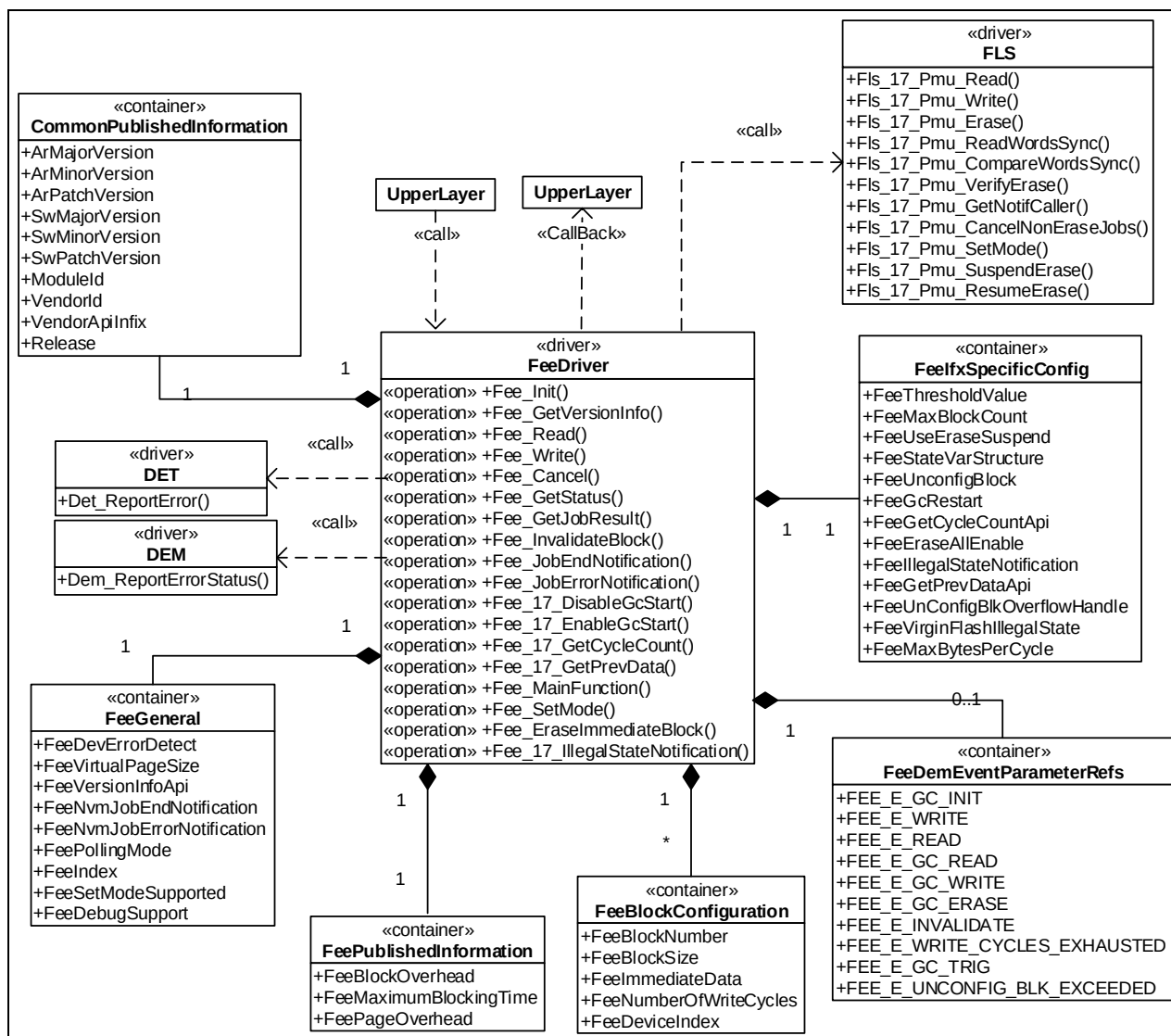


Figure 2 Interfaces of FEE driver

3 File Structure

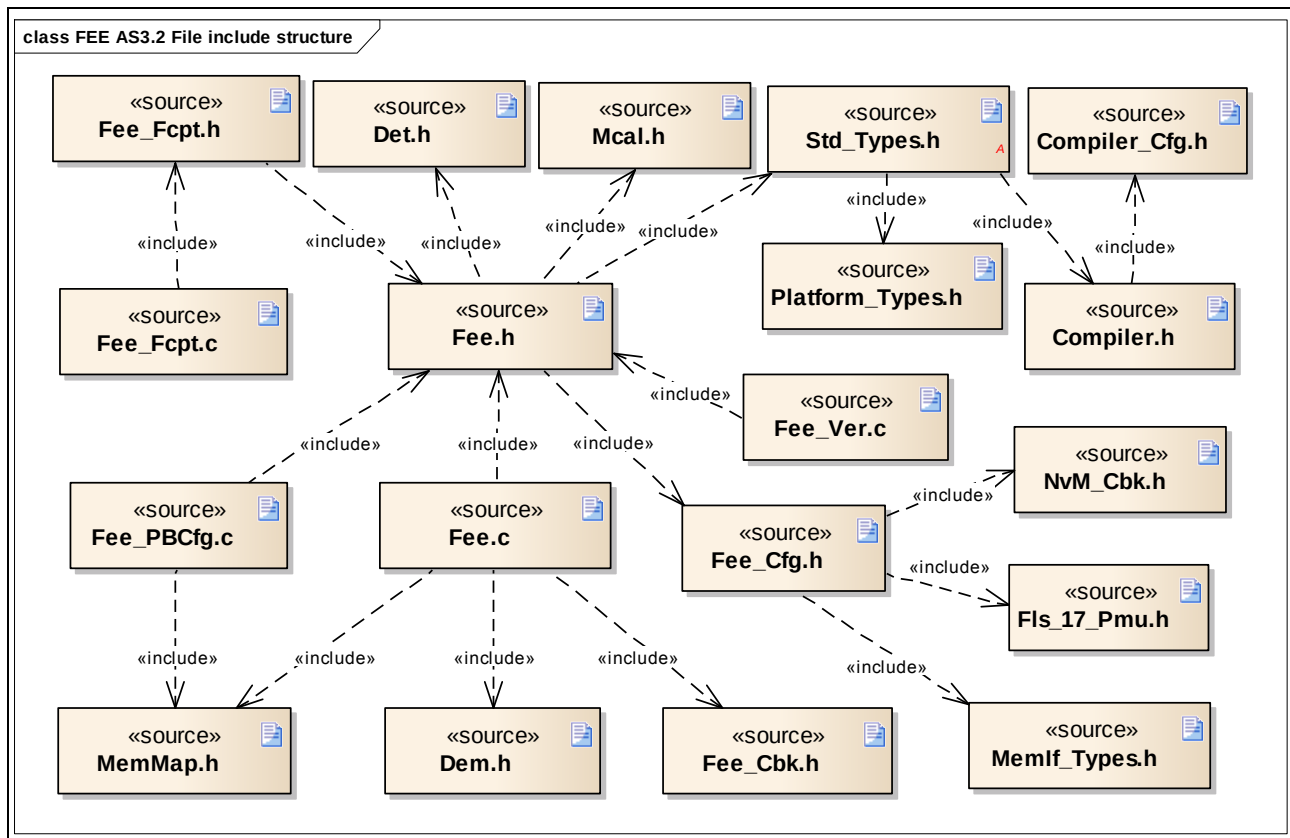


Figure 3 File structure of FEE driver.

Table 1 File Structure Contents

File Name	File Contents
Platform_Types.h	Platform dependencies are declared here
Compiler.h	Compiler dependencies are declared here
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code
Std_Types.h	Standard data types are declared here
MemIf_Types.h	Memory abstraction interface types are declared here
Fee.h	This header file exports the functionality of FEE driver
Fee.c	This file contains the functionality of FEE driver
Fee_Ver.c	Autosar version specific functionality of FEE driver
Fee_Cfg.h	Contains pre-compile configuration data of FEE driver
Fee_PBCfg.c	Contains configuration data of the FEE driver
Dem.h	This header file exports the functionality of DEM module
Det.h	This header file exports the functionality of DET module

MemMap.h	Mapping of code and data (variables, constants) to specific memory sections
Fls_17_Pmu.h	This header file exports the functionality of FLS driver
Fee_Cbk.h	This header file contains the declarations of callback functions provided by the FEE driver

4 Configuration Documentation

This chapter summarizes

- Configuration concept used for structuring the Driver
- Configuration parameter variance w.r.t supported HW
- All configuration parameters and their description

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and
- Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be **frozen before compilation** and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

Note: The FEE driver is delivered as Variant PC. However, to allow the usage of FEE driver across applications, few configuration parameters have been implemented as Post Build parameters. Hence the file Fee_PBCfg.c is part of the FEE driver.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

Note: Topics 4.1.1, 4.1.2 should be read for understanding the configuration class field

Table 3 Configuration Class Example 1

Configuration Class:	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, Variant PC.

Table 4 Configuration Class Example 2

Configuration Class:	Pre-compile	X	Variant PC
	Link Time	--	--
	Post build	X	Variant PB

'x' indicates that the parameter is implemented as a

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

4.2 Configuration Parameter Variance

This section describes the variance in the configuration parameter w.r.t supported microcontroller.

Refer section [4.4.2](#), [4.5.1](#) and [4.5.2](#).

4.3 Container: FeeGeneral

This container describes the global configuration for FEE.

4.3.1 FeeDevErrorDetect

Table 5 FeeDevErrorDetect

Name:	FeeDevErrorDetect		
Source:	This parameter is defined by AUTOSAR		
File:	Fee_Cfg.h		
Range:	Values: TRUE – development error detection and reporting enabled FALSE – development error detection and reporting disabled Default Value: FALSE		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable development error detection and reporting		
Scope:	Driver		
Dependency:	None		

4.3.2 FeeIndex

Table 6 FeeIndex

Name:	FeeIndex		
Source:	This parameter is defined by AUTOSAR		
File:	Fee_Cfg.h		
Range:	Values: 0 Default Value: 0		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Specifies the instace id of this module instance. Only one instance of FEE driver exists hence it is fixed to 0 (not configurable).		
Scope:	Driver		
Dependency:	None		

4.3.3 FeeNvmJobEndNotification

Table 7 FeeNvmJobEndNotification

Name:	FeeNvmJobEndNotification		
Source:	This parameter is defined by AUTOSAR		
File:	Fee_PBCfg.c		

Range:	Values: User defined notification function pointer Default Value: NULL_PTR		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	This parameter is the Job End notification function name and is called when the user requested asynchronous FEE job is successfully completed. The user must define a function by this name in his application. If no notification is required this parameter must be configured as "NULL_PTR". Notification function prototype: void <FeeNvmJobEndNotification> (void). Notification function is a Synchronous function.		
Scope:	Driver		
Dependency:	None		

4.3.4 FeeNvmJobErrorNotification

Table 8 FeeNvmJobErrorNotification

Name:	FeeNvmJobErrorNotification		
Source:	This parameter is defined by AUTOSAR		
File:	Fee_PBCfg.c		
Range:	Values: User defined notification function pointer Default Value: NULL_PTR		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	This parameter is the Job End notification function name and is called when the user requested asynchronous FEE job is failed. The user must define a function by this name in his application. If no notification is required this parameter must be configured as "NULL_PTR". Notification function prototype: void <FeeNvmJobErrorNotification> (void). Notification function is a Synchronous function.		
Scope:	Driver		
Dependency:	None		

4.3.5 FeePollingMode

Table 9 FeePollingMode

Name:	FeePollingMode
--------------	----------------

Source:	This parameter is defined by AUTOSAR		
File:	Fee_Cfg.h		
Range:	Values: TRUE – polling mode enabled FALSE – polling mode disabled Default Value: FALSE		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable the polling mode. As callback functions are always provided to the FLS module, FeePollingMode is always disabled, hence pre-configured as FALSE (not configurable).		
Scope:	Driver		
Dependency:	None		

4.3.6 FeeSetModeSupported

Table 10 FeeSetModeSupported

Name:	FeeSetModeSupported		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: TRUE - Fee_SetMode API enabled FALSE - Fee_SetMode API defined as an empty function Default value: FALSE		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable Fee_SetMode API.		
Scope:	Driver		
Dependency:	This configuration setting has to be consistent with that of the FLS driver (refer configuration parameter FlsSetModeApi of FLS driver).		

4.3.7 FeeVersionInfoApi

Table 11 FeeVersionInfoApi

Name:	FeeVersionInfoApi
Source:	This parameter is defined by AUTOSAR

File:	Fee_Cfg.h		
Range:	Values: TRUE - Fee_GetVersionInfo API enabled FALSE - Fee_GetVersionInfo API disabled Default value: FALSE		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Pre-processor switch to enable/disable Fee_GetVersionInfo API.		
Scope:	Driver		
Dependency:	None		

4.3.8 FeeVirtualPageSize

Table 12 FeeVirtualPageSize

Name:	FeeVirtualPageSize		
Source:	This parameter is defined by AUTOSAR		
File:	Fee_Cfg.h		
Range:	Values: 8 Default Value: 8		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Specifies the size in bytes to which the logical blocks shall be aligned. The value of this parameter is fixed (not configurable) to 8 i.e, page size of DF_EEPROM. Note: Deviation to AUTOSAR requirement [FEE076, FEE005]: (i) To find the physical address of the block, the FEE algorithm uses the cache table, i.e., the physical address of the block is not calculated based on the block number using any static equation. (ii) Due to the usage of double sector algorithm with implementation specific block management pages for each of the blocks, blocks will not be aligned as per the described relation between FeeBlockNumber and FeeVirtualPageSize. Hence, this configuration parameter is not significant for implementation.		
Scope:	Driver		
Dependency:	None		

4.4 Container: FeeBlockConfiguration

This container describes the logical block specific configurations.

4.4.1 FeeBlockNumber

Table 13 FeeBlockNumber

Name:	FeeBlockNumber		
Source:	This parameter is defined by the AUTOSAR		
File:	Fee_PBCfg.c		
Range:	Values: 1 to 65534 Default Value: 1		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	This parameter specifies the block number of the logical block. The value of this parameter should be unique in a given block configuration set. Note that, if a block is shared across block configuration sets, i.e., if the same block number is used across configuration sets, then all the configuration parameters of that block, should have same values. Note: To find the physical address of the block, the FEE algorithm uses the cache table, i.e., the physical address of the block is not calculated based on the block number using any static relation as suggested by AUTOSAR. Hence, the block number can be any unique arbitrary number.		
Scope:	Driver		
Dependency:	None		

4.4.2 FeeBlockSize

Table 14 FeeBlockSize

Name:	FeeBlockSize		
Source:	This parameter is defined by the AUTOSAR		
File:	Fee_PBCfg.c		
Range:	Values (in Bytes): 1 to 65535 (for TC27x) 1 to 42098 (for TC26x) 1 to 65535 (for TC29x) 1 to 56434 (for TC23x) 1 to 42098 (for TC22x) 1 to 20594 (for TC24x)		

	1 to 27762 (for TC21x) Default Value: 7		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	This parameter specifies the size of the logical block.		
Scope:	Driver		
Dependency:	None		

4.4.3 FeeImmediateData

Table 15 FeeImmediateData

Name:	FeeImmediateData		
Source:	This parameter is defined by AUTOSAR		
File:	Fee_PBCfg.c		
Range:	Values: TRUE - Immediate data block FALSE – Normal data block Default value: FALSE		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	This parameter specifies the type (priority) of the logical block		
Scope:	Driver		
Dependency:	None		

4.4.4 FeeNumberOfWriteCycles

Table 16 FeeNumberOfWriteCycles

Name:	FeeNumberOfWriteCycles		
Source:	This parameter is defined by the AUTOSAR		
File:	Fee_PBCfg.c		
Range:	Values: 0 to 16777215 Default Value: 0		
Configuration Class	Pre-Compile	--	--
	Link Time	--	--
	Post Build	X	All Variants
Description:	This parameter defines the block write cycle count of a particular logical block. It denotes the maximum number of times a particular logical block can be written.		

	However, the value 0 denotes that this block is not bound by any limit and can be written as many times as desired by the user.
Scope:	Driver
Dependency:	None

4.4.5 FeeDeviceIndex

Table 17 FeeDeviceIndex

Name:	FeeDeviceIndex		
Source:	This parameter is defined by the AUTOSAR		
File:	NA		
Range:	Reference to [Fls/FlsGeneral]		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter specifies the device index (handle). This information is needed by the NVRAM manager respectively the Memory Abstraction Interface to address a certain logical block.		
Scope:	Driver		
Dependency:	This information is needed by the NVRAM manager respectively the Memory Abstraction Interface to address a certain logical block.		

4.5 Container: FeeIfxSpecificConfig

This container describes the IFX specific configuration parameters required for FEE Driver.

4.5.1 FeeThresholdValue

Table 18 Specification of FeeThresholdValue

Syntax:	FeeThresholdValue
Source:	This parameter is defined by IFX
File:	Fee_PBCfg.c
Range:	Values: 0 to 195560 (for TC27x) 0 to 48104 (for TC26x) 0 to 392168 (for TC29x) 0 to 64488 (for TC23x) 0 to 48104 (for TC22x) 0 to 23528 (for TC24x) 0 to 31720 (for TC21x) Default value: 2048

Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter describes the threshold value (in bytes from the end of FEE sector) for triggering garbage collection/sector change. ----- Minimum size of threshold area shall be the sum of the following: - area required on flash for storing one copy of all the configured immediate blocks - area required on flash for storing the biggest immediate block - 512 bytes (i.e., size of one wordline) ----- Area required on flash for storing a block is calculated as follows: - 7 data bytes can be stored per page(8 bytes) - Additional 2 pages (16 bytes) for storing FEE internal block management data. Eg: If block size is 28 bytes: 4 + 2 pages = 48 bytes on flash If block size is 30 bytes: 5 + 2 pages = 56 bytes on flash ----- Calculation of minium threshold size to be configured: Block 1 – Normal block of size 50 bytes Block 2 – Normal block of size 60 bytes Block 3 – Immediate block of size 28 bytes Block 4 – Immediate block of size 30 bytes Only immediate block needs to be considered for threshold calculation: Block 3 – 28 bytes – needs 48 bytes in flash Block 4 – 30 bytes – needs 56 bytes in flash Hence minimum threshold size shall be: (48+56)+(56)+512=672 bytes ----- Note: - Higher value of threshold size can be configured to take care of interrupted garbage collection cases. - Higher the threshold size, higher will be the sector switch frequency. - Hence, threshold size shall be configured appropriately based on the application need.		
Scope:	Driver		
Dependency:	None		

4.5.2 FeeMaxBlockCount

Table 19 Specification of FeeMaxBlockCount

Syntax:	FeeMaxBlockCount		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: 1 to 8139 (for TC27x) 1 to 1995 (for TC26x) 1 to 16331 (for TC29x) 1 to 2678 (for TC23x) 1 to 1995 (for TC22x) 1 to 971 (for TC24x) 1 to 1312 (for TC21x) Default value: 1		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This configuration parameter specifies the total number of blocks throughout all configurations; Note that the blocks which are shared across configurations are counted only once. This parameter decides the cache table size at pre-compile time. The cache table holds the information of both configured and un-configured blocks. Example 1: Configuration sets having mutually exclusive blocks: Configuration A: Number of blocks configured is 10. Configuration B: Number of blocks configured is 25. Then, FeeMaxBlockCount = 35. Example 2: 5 blocks are shared/used by both Configuration A and Configuration B: Configuration A: Number of blocks configured is 10. Configuration B: Number of blocks configured is 25. Then, FeeMaxBlockCount = 30 (i.e., shared blocks are counted only once). Configuration of this parameter shall be carefully done. Higher value implies higher RAM area consumption for cache table. Lower value would result in loss of un-configured block(s) during GC (if FeeUnConfigBlkOverflowHandle is FEE_CONTINUE) or illegal state of		

	FEE (if FeeUnConfigBlkOverflowHandle is FEE_STOP_AT_GC). During cache build process if the detected un-configured blocks could not be accommodated in the cache table, then the error FEE_E_UNCONFIG_BLK_EXCEEDED is reported to DEM, if enabled. As a safety measure to avoid shortage of space in cache table at run-time, cache table size is determined such that it can accommodate ten more un-configured blocks in addition to the configured value. If FeeUnConfigBlkOverflowHandle is configured as FEE_CONTINUE: Un-configured blocks which could not be accommodated in the cache table, will be lost after the GC. FEE will continue as expected for the currently active configuration. If FeeUnConfigBlkOverflowHandle is configured as FEE_STOP_AT_GC: During GC, FEE will enter a pseudo illegal state where only read is allowed but write is not allowed (immediate block writes are allowed until end of the active sector).
Scope:	Driver
Dependency:	None

4.5.3 FeeUseEraseSuspend

Table 20 Specification of FeeUseEraseSuspend

Syntax:	FeeUseEraseSuspend		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: TRUE - Erase suspend feature is enabled FALSE – Erase suspend feature is disabled Default value: FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter specifies the usage of the erase suspend feature provided by the hardware. If it is configured as TRUE, then user read, write, invalidate requests are serviced during the erase operation of the GC. If it is configured as FALSE, then the user request will be kept pending till the erase operation is completed.		
Scope:	Driver		
Dependency:	This configuration setting has to be consistent with that of the FLS		

driver i.e., FeeUseEraseSuspend can be true only if FlsUseEraseSuspend is true (refer configuration parameter FlsUseEraseSuspend in FLS module).

4.5.4 FeeStateVarStructure

Table 21 Specification of FeeStateVarStructure

Syntax:	FeeStateVarStructure		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: Name of the pointer variable Default value: Fee_StateVar		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter is the name of the structure which would contain all the global variables of the FEE driver. Using this, the user can allocate the space for the variables at his behest to avoid any possible linking problems.		
Scope:	Driver		
Dependency:	None		

4.5.5 FeeUnConfigBlock

Table 22 Specification of FeeUnConfigBlock

Syntax:	FeeUnConfigBlock		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: FEE_UNCONFIG_BLOCK_IGNORE - Unconfigured blocks are ignored during GC FEE_UNCONFIG_BLOCK_KEEP – Unconfigured blocks are considered during GC Default value: FEE_UNCONFIG_BLOCK_IGNORE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter specifies whether unconfigured blocks should be copied to the new sector or should be ignored during GC.		
Scope:	Driver		
Dependency:	None		

4.5.6 FeeUnConfigBlkOverflowHandle

Table 23 Specification of FeeUnConfigBlkOverflowHandle

Syntax:	FeeUnConfigBlkOverflowHandle		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: FEE_CONTINUE – FEE operation continues ignoring unaccommodated unconfigured blocks. FEE_STOP_AT_GC – FEE operation stops at GC. Default value: FEE_CONTINUE		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter specifies the behavior of the FEE driver when the unconfigured blocks detected at run-time could not be accommodated in the un-configured block's cache table. FEE_CONTINUE: Un-configured blocks which could not be accommodated in the cache table, will be lost after the GC. FEE will continue as expected for the currently active configuration. FEE_STOP_AT_GC: During GC, FEE will enter a pseudo illegal state where only read is allowed but write is not allowed.		
Scope:	Driver		
Dependency:	None		

4.5.7 FeeGcRestart

Table 24 Specification of FeeGcRestart

Syntax:	FeeGcRestart		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: FEE_GC_RESTART_INIT – Restart GC at initialization. FEE_GC_RESTART_WRITE – Restart GC at first write Default value: FEE_GC_RESTART_INIT		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter specifies the GC restarting point.		
Scope:	Driver		

Dependency:	None
--------------------	------

4.5.8 FeeGetCycleCountApi

Table 25 Specification of FeeGetCycleCountApi

Syntax:	FeeGetCycleCountApi		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: TRUE - Fee_17_GetCycleCount API enabled FALSE - Fee_17_GetCycleCount API disabled Default value: FALSE		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the Fee_17_GetCycleCount API.		
Scope:	Driver		
Dependency:	None		

4.5.9 FeeEraseAllEnable

Table 26 Specification of FeeEraseAllEnable

Syntax:	FeeEraseAllEnable		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: TRUE - Both the FEE sectors shall be erased when illegal state is reached FALSE - FEE sectors shall not be erased when illegal state is reached Default value: FALSE		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter allows the user to configure if the sectors should be erased when FEE identifies an illegal state. When configured to TRUE, both the FEE sectors will be erased. In such a case FEE shall resume back to normal state but previous data (if any) cannot be recovered. When configured to FALSE, FEE settles in the illegal state and cannot continue normally. However, data is kept intact in DFLASH and may be recovered.		

Scope:	Driver
Dependency:	None

4.5.10 FeeGetPrevDataApi

Table 27 Specification of FeeGetPrevDataApi

Syntax:	FeeGetPrevDataApi		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: TRUE - Fee_17_GetPrevData API enabled FALSE - Fee_17_GetPrevData API disabled Default value: FALSE		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Pre-processor switch to enable/disable the Fee_17_GetPrevData API.		
Scope:	Driver		
Dependency:	None		

4.5.11 FeeIllegalStateNotification

Table 28 Specification of FeeIllegalStateNotification

Syntax:	FeeIllegalStateNotification		
Source:	This parameter is defined by IFX		
File:	Fee_PBCfg.c		
Range:	Values: Pointer to FEE illegal state notification callback function Default value: NULL_PTR		
Configuration class	Pre-compile	--	--
	Link Time	--	--
	Post build	x	All Variants
Description:	This parameter specifies the pointer to the callback function, which will be called when the FEE reaches the illegal state.		
Scope:	Driver		
Dependency:	None		

4.5.12 FeeVirginFlashIllegalState

Table 29 Specification of FeeVirginFlashIllegalState

Syntax:	FeeVirginFlashIllegalState
----------------	----------------------------

Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: TRUE: (only if FeeEraseAllEnable is FALSE) Upon detection of virgin flash (DF_EEPROM) - FEE reaches illegal state and calls the configured illegal state notification function. FALSE: Upon detection of virgin flash (DF_EEPROM) - FEE programs the initial state pages and will be available for user requests Default value: FALSE		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	This parameter allows the user to configure the behavior of FEE upon detection of virgin Flash (DF_EEPROM). It can be either configured to reach illegal state (only if FeeEraseAllEnable is FALSE) and call the configured illegal state notification function OR to program the state pages and perform normal operation.		
Scope:	Driver		
Dependency:	FeeEraseAllEnable has higher priority than this parameter i.e., if FeeEraseAllEnable is TRUE then FEE programs the initial state pages and will be available for user requests irrespective of the configured value of FeeVirginFlashIllegalState. If FeeEraseAllEnable is FALSE then this parameter will work as described.		

4.5.13 FeeMaxBytesPerCycle

Table 30 Specification of FeeMaxBytesPerCycle

Syntax:	FeeMaxBytesPerCycle		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: FEE_MAX_BYTES_64 – Maximum of 64 bytes in one cycle FEE_MAX_BYTES_128 – Maximum of 128 bytes in one cycle FEE_MAX_BYTES_256 – Maximum of 256 bytes in one cycle FEE_MAX_BYTES_512 – Maximum of 512 bytes in one cycle Default value: FEE_MAX_BYTES_64		
Configuration class	Pre-compile	x	--
	Link Time	--	--
	Post build	--	All Variants
Description:	This parameter specifies the maximum number of data bytes that will be processed in one FEE Main function call during read, write and compare operations of FEE.		

Scope:	Driver
Dependency:	None

4.6 Container: FeePublishedInformation

4.6.1 FeeBlockOverhead

Table 31 FeeBlockOverhead

Name:	FeeBlockOverhead		
Source:	This parameter is defined by the AUTOSAR		
File:	Fee_Cfg.h		
Range:	Value (fixed): 17		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	Block management overhead per logical block in bytes is given by: (i) If $((\text{FeeBlockSize} \% 7) == 0)$: $\rightarrow ((\text{FeeBlockSize} / 7) * 8) + 16 - \text{FeeBlockSize}$ (ii) If $((\text{FeeBlockSize} \% 7) != 0)$: $\rightarrow (((\text{FeeBlockSize} / 7) + 1) * 8) + 16 - \text{FeeBlockSize}$ Note: Integer arithmetic is used in the above calculation.		
Scope:	Driver		
Dependency:	None		

4.6.2 FeeMaximumBlockingTime

Table 32 FeeMaximumBlockingTime

Name:	FeeMaximumBlockingTime		
Source:	This parameter is defined by the AUTOSAR		
File:	Fee_Cfg.h		
Range:	Value (in ms): 10		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	The maximum time (in ms) the FEE module's API routines shall be blocked (delayed) by internal operations. Note: The following scenarios may result in a exceeding the value specified. Here, job execution delay is mentioned and not the blocking time of the API (i.e., API will not be blocked without returning):		

	(i) During initialization, for the configuration FEE_GC_RESTART_WRITE: For the first FEE job after Fee_Init, time would be consumed for reading the state pages as well as for updating the cache table. A write request will be pending until this time. (ii) During GC copy state: Start of the immediate block's write operation will be delayed until the current block being copied is completely copied. So, the worst case depends upon the maximum block size configured. (iii) If erase suspend feature is not enabled then the user request's execution will be delayed until the FEE sector is erased.
Scope:	Driver
Dependency:	None

4.6.3 FeePageOverhead

Table 33 FeePageOverhead

Name:	FeePageOverhead		
Source:	This parameter is defined by the AUTOSAR		
File:	Fee_Cfg.h		
Range:	Value (fixed): 1		
Configuration Class	Pre-Compile	X	All Variants
	Link Time	--	--
	Post Build	--	--
Description:	This parameter specifies the page overhead in bytes. This value is applicable only for the pages containing logical block data bytes (i.e., it is not applicable for the pages which contain only implementation specific management data).		
Scope:	Driver		
Dependency:	None		

4.7 FeeDemConfig

If this container does not exist then, production error checking and reporting will be enabled by default.

4.7.1 FeeGcInitDemReport

Table 34 Specification of FeeGcInitDemReport

Syntax:	FeeGcInitDemReport
Source:	This parameter is defined by IFX
File:	Fee_Cfg.h
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM

	DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error during Init GC activity.		
Scope:	Driver		
Dependency:	None		

4.7.2 FeeWriteDemReport

Table 35 Specification of FeeWriteDemReport

Syntax:	FeeWriteDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error while executing user write request		
Scope:	Driver		
Dependency:	None		

4.7.3 FeeReadDemReport

Table 36 Specification of FeeReadDemReport

Syntax:	FeeReadDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		

Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Uncorrectable ECC error while executing user read request		
Scope:	Driver		
Dependency:	None		

4.7.4 FeeGCReadDemReport

Table 37 Specification of FeeGCReadDemReport

Syntax:	FeeGCReadDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Uncorrectable ECC error while reading from DFlash during GC.		
Scope:	Driver		
Dependency:	None		

4.7.5 FeeGCWriteDemReport

Table 38 Specification of FeeGCWriteDemReport

Syntax:	FeeGCWriteDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		

Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error while executing write operation during GC.		
Scope:	Driver		
Dependency:	None		

4.7.6 FeeGCEraseDemReport

Table 39 Specification of FeeGCEraseDemReport

Syntax:	FeeGCEraseDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error while executing erase operation during GC.		
Scope:	Driver		
Dependency:	None		

4.7.7 FeeInvalidateDemReport

Table 40 Specification of FeeInvalidateDemReport

Syntax:	FeeInvalidateDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--

	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error while executing user triggered invalidate request.		
Scope:	Driver		
Dependency:	None		

4.7.8 FeeWriteCyclesDemReport

Table 41 Specification of FeeWriteCyclesDemReport

Syntax:	FeeWriteCyclesDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error due to exceeding the configured limit of write cycles for the given block.		
Scope:	Driver		
Dependency:	None		

4.7.9 FeeGcTrigDemReport

Table 42 Specification of FeeGcTrigDemReport

Syntax:	FeeGcTrigDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--

	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Error due to insufficient space in the new sector for copying data blocks or state page.		
Scope:	Driver		
Dependency:	None		

4.7.10 FeeUnconfigBlkExceededDemReport

Table 43 Specification of FeeUnconfigBlkExceededDemReport

Syntax:	FeeUnconfigBlkExceededDemReport		
Source:	This parameter is defined by IFX		
File:	Fee_Cfg.h		
Range:	Values: ENABLE_DEM_REPORT – Error will be reported to DEM DISABLE_DEM_REPORT – Error will not be reported to DEM Default value: ENABLE_DEM_REPORT		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	Error will be reported to DEM, either if this parameter does not exist or if this parameter exists and configured as ENABLE_DEM_REPORT. Error description: Unable to accommodate the detected un-configured block.		
Scope:	Driver		
Dependency:	None		

5 Published Parameters

This section describes the parameters published by this FEE driver.

5.1 Vendor ID

Table 44 Vendor ID

Syntax:	FEE_VENDOR_ID
Type:	Macro / uint16
File:	Fee_Cfg.h
Value:	17
Description:	Vendor is Infineon Technologies

5.2 Module ID

Table 45 Module ID

Syntax:	FEE_MODULE_ID
Type:	Macro / uint16
File:	Fee_Cfg.h
Value:	21
Description:	This macro gives the FEE driver module ID as described by AUTOSAR

5.3 AR Major Version

Table 46 FEE Driver AUTOSAR Requirement Major Version

Syntax:	FEE_AR_MAJOR_VERSION
Type:	Macro
File:	Fee_Cfg.h
Value:	Refer Fee_Cfg.h
Description:	Major version number of AUTOSAR specification on which the driver implementation is based on.

5.4 AR Minor Version

Table 47 FEE Driver AUTOSAR Requirement Minor Version

Syntax:	FEE_AR_MINOR_VERSION
Type:	Macro
File:	Fee_Cfg.h
Value:	Refer Fee_Cfg.h

Description:	Minor version number of AUTOSAR specification on which the driver implementation is based on.
---------------------	---

5.5 AR Revision Version

Table 48 FEE Driver AUTOSAR Requirement Revision Version

Syntax:	FEE_AR_PATCH_VERSION
Type:	Macro
File:	Fee_Cfg.h
Value:	Refer Fee_Cfg.h
Description:	Revision version number of AUTOSAR specification on which the driver implementation is based on.

5.6 SW Major Version

Table 49 SW Major Version

Syntax:	FEE_SW_MAJOR_VERSION
Type:	Macro
File:	Fee_Cfg.h
Value:	Refer Fee_Cfg.h
Description:	Major version number of the vendor specific implementation of the Driver.

5.7 SW Minor Version

Table 50 SW Minor Version

Syntax:	FEE_SW_MINOR_VERSION
Type:	Macro
File:	Fee_Cfg.h
Value:	Refer Fee_Cfg.h
Description:	Minor Version Number of the vendor specific implementation of the Driver.

5.8 SW Patch Version

Table 51 SW Patch Version

Syntax:	FEE_SW_PATCH_VERSION
Type:	Macro
File:	Fee_Cfg.h
Value:	Refer Fee_Cfg.h

Syntax:	FEE_SW_PATCH_VERSION
Description:	Patch Version Number of the vendor specific implementation of the Driver.

6 API Documentation

6.1 API Type Definitions

6.1.1 Type Definition Fee_ConfigType

Table 52 Type Definition Fee_ConfigType

Syntax:	Fee_ConfigType
Type:	struct
File:	Fee.h
Range:	This datatype is used for the configuration of the entire FEE Driver
Description:	It contains the configuration for FEE and also the driver notification settings.

6.2 API Functions

This chapter describes the API interfaces provided by FEE Driver.

6.2.1 Fee_Init

Table 53 Specification of Fee_Init

Service name:	Fee_Init
Syntax:	void Fee_Init (void)
Service ID:	0x00
Sync/ Async:	Synchronous
Origin	Autosar
Re-entrancy:	non reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This service initializes the FEE driver as per the configuration. The FEE Driver variables are initialized. Also the state pages of both the sectors are read and the GC state is determined.
Caveats:	This service should not be called during a running operation.
Configuration:	None

DET:	None	
DEM:	None	
Implementation Comments:	If FeeGcRestart is configured to FEE_GC_RESTART_INIT then this API also requires the Fee_MainFunction to initiate "InitGC", "Cache update". Refer section 6.3.1. <i>In FEE_GC_RESTART_INIT configuration, a GC is restarted by the Fee_MainFunction after FEE initialization in case GC was started in the previous run but interrupted before completion. The detection of the GC restart condition is evaluated on the basis of the state page value only. The filling level of the sector is not taken into account.</i> <i>In case that the sector is filled over the threshold value (immediate data blocks are written and GC had not started before interruption), no GC is executed by the Fee_MainFunction after FEE initialization even if FEE_GC_RESTART_INIT is configured. The GC is started with the next Fee_Write/Fee_Invalidate function call.</i>	
I/O Mode:	User Mode	

6.2.2 Fee_SetMode

Table 54 Specification of Fee_SetMode

Service name:	Fee_SetMode	
Syntax:	<pre>void Fee_SetMode (MemIf_ModeType Mode)</pre>	
Service ID:	0x01	
Sync/ Async:	Synchronous	
Orgin	Autosar	
Re-entrancy:	non reentrant	
Parameters (in):	Mode	Desired mode for the underlying flash driver
Parameters (out):	None	
Return value:	None	
Description:	Service to set the mode of the underlying FLS driver	
Caveats:	None	
Configuration:	FeeSetModeSupported = TRUE	
DET:	None	
DEM:	None	
Implementation Comments:	As per the FLS driver, FAST/SLOW modes are applicable only for read operations, i.e., it has no effect on write operations.	

I/O Mode:	User Mode
-----------	-----------

6.2.3 Fee_Read

Table 55 Specification of Fee_Read

Service name:	Fee_Read	
Syntax:	<pre>Std_ReturnType Fee_Read (uint16 BlockNumber, uint16 BlockOffset, uint8 *DataBufferPtr, uint16 Length)</pre>	
Service ID:	0x02	
Sync/ Async:	Asynchronous	
Origin	Autosar	
Re-entrancy:	Non re-entrant	
Parameters (in):	BlockNumber	Logical block number
	BlockOffset	Read address offset within the block
	Length	Number of bytes to be read
Parameters (out):	DataBufferPtr	Pointer to data buffer
Return value:	E_OK	The read job is accepted
	E_NOT_OK	The read job has not been accepted.
Description:	It performs the DET checks to ascertain the validity of the request and then registers the request to reads the block contents from DF_EEPROM only if no other user request is pending.	
Caveats:	None	
Configuration:	None	
DET:	FEE_E_UNINIT	Fee_Init API was not called before executing this request
	FEE_E_INVALID_BLOCK_NO	Block number does not exist in the configuration
	FEE_E_INVALID_BLOCK_OFS	Block Offset is invalid
	FEE_E_INVALID_DATA_PTR	DataBufferPtr is NULL_PTR
	FEE_E_INVALID_BLOCK_LEN	BlockOffset + Length > Size of the block (as in configuration)
	FEE_E_BUSY	The read request cannot be

		accepted as FEE is already busy with a previous request
DEM:	Errors are reported asynchronously through Fee_JobErrorNotification	
Implementation Comments:	If FeeGcRestart is configured to FEE_GC_RESTART_WRITE, the first call of this API from upper layer (provided there were no write/invalidate requests earlier) will request the Fee_MainFunction to initiate "InitGC", "Cache update". Refer section 6.3.10 . <i>Note: However the resumption of GC is NOT triggered.</i>	
I/O Mode:	User Mode	

6.2.4 Fee_Write

Table 56 Specification of Fee_Write

Service name:	Fee_Write	
Syntax:	Std_ReturnType Fee_Write (uint16 BlockNumber, uint8 *DataBufferPtr) 	
Service ID:	0x03	
Sync/ Async:	Asynchronous	
Origin	Autosar	
Re-entrancy:	Non re-entrant	
Parameters (in):	BlockNumber	Logical block number
	DataBufferPtr	Pointer to data buffer
Parameters (out):	None	
Return value:	E_OK	The write job is accepted.
	E_NOT_OK	The write job has not been accepted.
Description:	It performs the DET checks to ascertain the validity of the request and then registers the request to write the desired block contents to DF_EEPROM only if no other user request is pending. <i>Note: A job can fail due to hardware errors or data verification failure (>1 per block)</i> <i>Note: To check the status of job completion, Fee_GetJobResult should be used</i> <i>Note: As the data is written in pages or group of 4 pages, incase only a part of the page is used for the block data,</i>	

	<i>the remaining bytes of the page will be filled with 0xFF</i>	
Caveats:	None	
Configuration:	None	
DET:	FEE_E_UNINIT	Fee_Init API was not called before executing this request
	FEE_E_INVALID_BLOCK_NO	Block number does not exist in the configuration
	FEE_E_INVALID_DATA_PTR	DataBufferPtr is NULL_PTR
	FEE_E_BUSY	The write request cannot be accepted as FEE is already busy with a previous request
DEM:	Errors are reported asynchronously through Fee_JobErrorNotification	
Implementation Comments:	If FeeGcRestart is configured to FEE_GC_RESTART_WRITE, the first call of this API from upper layer (provided there were no read/invalidate requests earlier) will request the Fee_MainFunction to initiate "InitGC", "Cache update". Refer section 6.3.10 .	
I/O Mode:	User Mode	

6.2.5 Fee_Cancel

Table 57 Specification of Fee_Cancel

Service name:	Fee_Cancel
Syntax:	void Fee_Cancel (void)
Service ID:	0x04
Sync/ Async:	Asynchronous
Origin	Autosar
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This function resets the module internal state machine and prepares the module to accept the next job request. If a write/invalidate request is ongoing then the cancellation is not done. In all the other cases, the request is canceled.
Caveats:	None
Configuration:	None

DET:	FEE_E_UNINIT : Fee_Init API was not called before executing this request FEE_E_INVALID_CANCEL: If there was no on-going user job
DEM:	None
Implementation Comments:	None
I/O Mode:	User Mode

6.2.6 Fee_GetStatus

Table 58 Specification of Fee_GetStatus

Service name:	Fee_GetStatus	
Syntax:	MemIf_StatusType Fee_GetStatus (void)	
Service ID:	0x05	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	MEMIF_UNINIT	The FEE driver has not been initialized (Fee_Init not called)
	MEMIF_IDLE	The FEE Driver and the underlying flash driver are currently idle. This can happen in the following cases 1. When there is no pending request and GC is idle 2. When there is no pending request and GC cannot be started because GC restart point is FEE_GC_RESTART_WRITE 3. When there is no pending request and GC has entered a fail state 4. When there is no pending request and Init GC has

		entered a fail state 5. When the write has entered a fail state (compare failed twice / HW error on a previously programmed block)
	MEMIF_BUSY	The FEE Driver and the underlying flash driver are currently busy.
	MEMIF_BUSY_INTERNAL	The FEE module is busy with internal management operations.
Description:	This function returns the FEE driver status	
Caveats:	None	
Configuration:	None	
DET:	FEE_E_UNINIT : Fee_Init API was not called before executing this request	
DEM:	None	
Implementation Comments:	None	
I/O Mode:	User Mode	

6.2.7 Fee_GetJobResult

Table 59 Specification of Fee_GetJobResult

Service name:	Fee_GetJobResult	
Syntax:	MemIf_JobResultType Fee_GetJobResult (void)	
Service ID:	0x06	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Non re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	MEMIF_JOB_OK	The last job has been finished successfully.
	MEMIF_JOB_PENDING	The last job is waiting for execution or currently being

		executed. This can happen in the following cases 1. When there is a pending request waiting to be executed (GC/Init on going) 2. When there is a pending request being executed 3. When there is a pending request and FEE entered a fail state because of internal operation (compare failed twice / HW error during Init GC, GC, re-write of a previously programmed block)
	MEMIF_JOB_CANCELED	The last job has been canceled.
	MEMIF_JOB_FAILED	1) The FEE driver has not been initialized (Fee_Init not called) 2) The last read/write/invalidate job failed.
	MEMIF_BLOCK_INCONSISTENT	The requested block to be read is present in the configuration but is not written in DF_EEPROM yet
	MEMIF_BLOCK_INVALID	The requested block has been invalidated hence, the requested read operation cannot be performed.
Description:	This function returns the job result of the ongoing/previous Job.	
Caveats:	None	
Configuration:	None	

DET:	FEE_E_UNINIT: Fee_Init API was not called before executing this request
DEM:	None
Implementation Comments:	None
I/O Mode:	User Mode

6.2.8 Fee_InvalidateBlock

Table 60 Specification of Fee_InvalidateBlock

Service name:	Fee_InvalidateBlock	
Syntax:	Std_ReturnType Fee_InvalidateBlock (uint16 BlockNumber)	
Service ID:	0x07	
Sync/ Async:	Asynchronous	
Origin	Autosar	
Re-entrancy:	Non re-entrant	
Parameters (in):	BlockNumber	Logical block number
Parameters (out):	None	
Return value:	E_OK	The job is accepted.
	E_NOT_OK	The job has not been accepted.
Description:	It performs the DET checks to ascertain the validity of the request and then registers the request to invalidate the block. The request is registered only if no other user request is pending. <i>Note: A job can fail due to hardware errors or data verification failure (>1 per block) To check the status of job completion, Fee_GetJobResult should be used</i>	
Caveats:	None	
Configuration:	None	
DET:	FEE_E_UNINIT	Fee_Init API was not called before executing this request
	FEE_E_INVALID_BLOCK_NO	Block number does not exist in the configuration
	FEE_E_BUSY	The invalidate request cannot be accepted as FEE is already busy with a previous request
DEM:	Errors are reported asynchronously through	

	Fee_JobErrorNotification
Implementation Comments:	If FeeGcRestart is configured to FEE_GC_RESTART_WRITE, the first call of this API from upper layer (provided there were no read/write requests earlier) will request the Fee_MainFunction to initiate "InitGC", "Cache update". Refer section 6.3.10 .
I/O Mode:	User Mode

6.2.9 Fee_GetVersionInfo

Table 61 Specification of Fee_GetVersionInfo

Service name:	Fee_GetVersionInfo	
Syntax:	<pre>void Fee_GetVersionInfo (Std_VersionInfoType *VersionInfoPtr)</pre>	
Service ID:	0x08	
Sync/ Async:	Synchronous	
Origin	Autosar	
Re-entrancy:	Re-entrant	
Parameters (in):	None	
Parameters (out):	VersionInfoPtr	Pointer to version information set.
Return value:	None	
Description:	This service provides the version information of this module.	
Caveats:	None	
Configuration:	FeeVersionInfoApi = TRUE	
DET:	If VersionInfoPtr is NULL_PTR, then the development error FEE_E_INVALID_DATA_PTR will be reported to DET.	
DEM:	None	
Implementation Comments:	This function is implemented as a macro	
I/O Mode:	User Mode	

6.2.10 Fee_EraseImmediateBlock

Table 62 Specification of Fee_EraseImmediateBlock

Service name:	Fee_EraseImmediateBlock	
Syntax:	<pre>Std_ReturnType Fee_EraseImmediateBlock(uint16 BlockNumber)</pre>	
Service ID:	0x09	
Sync/ Async:	Synchronous	

Origin	Autosar	
Re-entrancy:	non re-entrant	
Parameters (in):	BlockNumber	Logical block number
Parameters (out):	None	
Return value:	E_NOT_OK	
Description:	Since double-sector algorithm is used with threshold limit for triggering GC, immediate block's write requests during GC can be accommodated within the pre-erased threshold area of the active FEE sector. Hence, the API Fee_EraseImmediateBlock is implemented as a dummy function returning E_NOT_OK.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	This API returns E_NOT_OK as the functionality is not implemented	
I/O Mode:	User Mode	

6.2.11 Fee_JobEndNotification

Table 63 Specification of Fee_JobEndNotification

Service name:	Fee_JobEndNotification
Syntax:	<pre>void Fee_JobEndNotification (void)</pre>
Service ID:	0x10
Sync/ Async:	Synchronous
Origin	Autosar
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This routine is called by the underlying flash driver to report the successful end of an asynchronous operation. This routine performs necessary block management operations before calling the corresponding callback routine of the upper layer module.
Caveats:	None
Configuration:	None

DET:	None
DEM:	None
Implementation Comments:	This function directly returns to the caller incase Fee_Init API was not called before executing this function
I/O Mode:	User Mode

6.2.12 Fee_JobErrorNotification

Table 64 Specification of Fee_JobErrorNotification

Service name:	Fee_JobErrorNotification
Syntax:	void Fee_JobErrorNotification (void)
Service ID:	0x11
Sync/ Async:	Synchronous
Origin	Autosar
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This routine is called by the underlying flash driver to report the failure of an asynchronous operation. This routine performs any necessary block management and error handling operations before calling the corresponding callback routine of the upper layer module.
Caveats:	None
Configuration:	None
DET:	None
DEM:	FEE_E_WRITE, FEE_E_INVALIDATE, FEE_E_READ
Implementation Comments:	This function directly returns to the caller incase Fee_Init API was not called before executing this function
I/O Mode:	User Mode

6.2.13 Fee_17_GetCycleCount

Table 65 Specification of Fee_17_GetCycleCount

Service name:	Fee_17_GetCycleCount
Syntax:	Std_ReturnType Fee_17_GetCycleCount(uint16 BlockNumber,

	uint32 * const CountPtr)	
Service ID:	None	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non re-entrant	
Parameters (in):	BlockNumber	Logical block number
	CountPtr	Pointer to the counter variable
Parameters (out):	None	
Return value:	E_OK	Cycle Count read was successful
	E_NOT_OK	1. Could not read Cycle count value as the underlying Flash driver was busy 2. Read error was encountered while reading the cycle count from DF_EEPROM 3. Cache update activity is not yet complete 4. GC is ongoing
Description:	When called with BlockNumber = 0 this routine delivers the erase cycle count. When called with a configured BlockNumber, the write cycle count of the given block is returned.	
Caveats:	The BlockNumber shall be a valid number or 0. The pointer passed to the API shall be a valid pointer.	
Configuration:	FeeGetCycleCountApi = TRUE	
DET:	None	
DEM:	None	
Implementation Comments:	In case of GC_RESTART_WRITE configuration, the Fee_17_GetCycleCount returns E_NOT_OK until the first Fee Read / Write / Invalidate job is executed. This is because in GC_RESTART_WRITE configuration, the cache update is triggered by Fee Read / Write / Invalidate request. Refer section 6.3.10 .	
I/O Mode:	User Mode	

6.2.14 Fee_17_DisableGcStart

Table 66 Specification of Fee_17_DisableGcStart

Service name:	Fee_17_DisableGcStart
Syntax:	void Fee_17_DisableGcStart(void)
Service ID:	None
Sync/ Async:	Synchronous
Origin	Vendor specific
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	The user can call this API to prevent the GC from being started in case the threshold is crossed in the active sector. This API does not stop an ongoing GC but only prevents the GC from being triggered by the Write/Invalidate request issued by the user. This API will disable the GC from starting only if 1. GC is not ongoing 2. Initialisation activities are complete (i.e. the state pages are read and the GC state is determined) Refer section 6.3.3. The above activities are required to be completed for disabling of GC.
Caveats:	FEE driver initialization must be done before calling this API.
Configuration:	None
DET:	None
DEM:	None
Implementation Comments:	None
I/O Mode:	User Mode

6.2.15 Fee_17_EnableGcStart

Table 67 Specification of Fee_17_EnableGcStart

Service name:	Fee_17_EnableGcStart
Syntax:	void Fee_17_EnableGcStart(void)
Service ID:	None
Sync/ Async:	Synchronous

Origin	Vendor specific
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	This service allows enabling the trigger of GC, if GC was disabled earlier by calling the API Fee_17_DisableGcStart. After this API is called, if the sector is filled up to the threshold level and additional write / invalidate request is issued, then GC will be initiated. Refer section 6.3.3.
Caveats:	FEE driver initialization must be done before calling this API.
Configuration:	None
DET:	None
DEM:	None
Implementation Comments:	None
I/O Mode:	User Mode

6.2.16 Fee_MainFunction

Table 68 Specification of Fee_MainFunction

Service name:	Fee_MainFunction
Syntax:	void Fee_MainFunction (void)
Service ID:	None
Sync/ Async:	Synchronous
Origin	Autosar
Re-entrancy:	Non re-entrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	Fee_MainFunction state machine identifies the job to be executed (based on user request, internal status) and then executes the job. The various jobs handled by Fee_MainFunction are A. Internal jobs 1. GC: Switching data to new sector and erasing the old sector. 2. InitGC: First job to be done before executing the user requests. Involves detection of any interruption in the GC

	process during the previous ECU run. Also takes care of repairing state pages and determines resumption point of GC. 3. Cache update: Involves building a cache table to track the latest copy and previous copy of all blocks and their status B. User jobs 1. Read job: Handling of user read request 2. Write/Invalidate job: Handling of user write/invalidate request
Caveats:	None
Configuration:	None
DET:	None
DEM:	All DEMs mentioned in section 8
Implementation Comments:	None
I/O Mode:	User Mode

6.2.17 Fee_17_GetPrevData

Table 69 Specification of Fee_17_GetPrevData

Service name:	Fee_17_GetPrevData	
Syntax:	Std_ReturnType Fee_17_GetPrevData(uint16 BlockNumber, uint16 BlockOffset, uint8 *DataBufferPtr, uint16 Length)	
Service ID:	None	
Sync/ Async:	Asynchronous	
Origin	Vendor specific	
Re-entrancy:	Non re-entrant	
Parameters (in):	BlockNumber	Logical block number
	BlockOffset	Address offset within the block
	Length	Number of bytes to be read
Parameters (out):	DataBufferPtr	Pointer to data buffer
Return value:	E_OK	The read job is accepted
	E_NOT_OK	The read job has not been accepted.
Description:	The user can call this API to read one occurrence old data of the given block. This API accepts the request and updates the FEE internal variables.	

	However the actual reading of the old data is done by the Fee_MainFunction after the cache table is built. The result of this job can be queried using Fee_GetJobResult API.
Caveats:	All the parameters passed to the API shall be valid as there will not be any check for development error
Configuration:	FeeGetPrevDataApi = TRUE
DET:	None
DEM:	Errors are reported asynchronously through Fee_JobErrorNotification
Implementation Comments:	None
I/O Mode:	User Mode

6.2.18 Fee_17_IllegalStateNotification

Table 70 Specification of Fee_17_IllegalStateNotification

Service name:	Fee_17_IllegalStateNotification	
Syntax:	void Fee_17_IllegalStateNotification(void)	
Service ID:	None	
Sync/ Async:	Synchronous	
Origin	Vendor specific	
Re-entrancy:	Non re-entrant	
Parameters (in):	None	
Parameters (out):	None	
Return value:	void	
Description:	This routine shall be called by the underlying flash driver when it reaches the illegal state.	
Caveats:	None	
Configuration:	None	
DET:	None	
DEM:	None	
Implementation Comments:	When the underlying FLS driver reaches illegal state, this API shall be called i.e., FlsIllegalStateNotification shall be configured as Fee_17_IllegalStateNotification. This API calls the illegal state notificaiton function configured via FeeIllegalStateNotification. Upon reaching the illegal state, the upper layer may call the FLS driver API Fls_17_Pmu_GetOperStatus to check whether the illegal state is reached due to OPER error. Upon reaching the illegal state, the upper layer shall not call	

	any further FLS/FEE APIs. Reset of the system is recommended.
I/O Mode:	User Mode

6.3 Fee Feature Description

6.3.1 Emulation Algorithm

The chosen emulation algorithm is commonly called as “double-sector” algorithm. Although the entire DF_EEPROM area belongs to the same physical sector, the DF_EEPROM area is logically divided into two parts for the implementation of “double-sector” algorithm, each of which is referred as “FEE sector”. In this document “sector” refers to “FEE sector”, unless specified explicitly as “physical sector”.

In principle the algorithm defines two kinds of pages, they are [i]data pages, which basically stores the data requested by the user (which consists of data as well as implementation specific block management data); [ii]state pages (each state page comprises of a group of consecutive physical pages), which contains information about the state of the FEE sector (i.e.Valid/Erased).

Each data block is composed of multiple data pages based on the size of the block. These data blocks are written sequentially in the sector in the form of a stack, when requested by the user. Data is updated by simply appending the new data block to the end of this “stack”. When the stack is filled up to a certain threshold, garbage collection (GC) will be started.

GC is an internal procedure which is composed of the following steps:

1. Copy the required data from the active sector (filled sector) to the other sector (new sector),
1. Perform the sector switch i.e. the new sector now becomes active sector. This is also referred to as “End of Copy procedure”
2. Erase the filled sector.
3. Verify the erased sector and mark the “End of Erase procedure”. Verification involves detection of any unerased wordlines.

The state pages contain state information to identify the active sector (sector containing the latest valid data) and secondly it protects the long lasting garbage collection from unplanned resets or shutdowns.

The state pages are written as a part of the Garbage Collection to mark two important events:

1. End of Copy procedure i.e. all the required blocks are moved from the active sector (filled sector) to the new sector. This is denoted by writing the “VALID” state page in new sector.
2. End of Erase procedure i.e. the filled sector is erased and verified (i.e. checks for an unerased wordline and marks it). This is denoted by writing the “ERASED” state page in the sector that was erased.

Initially the FEE driver reads both the sectors to identify their state. Using the individual sector states, it determines the active sector and also the state of the garbage collection referred to as the GC state. This process is referred to as “InitGC”. The steps of InitGC can be summarized as:

1. Determination of the active sector and resumption point of GC
2. Repair/Rewrite of dirty state pages, if required
3. Erasure and/or preparation of DFLASH, if required
4. Resumption of GC, if required.

The next step is to read the active sector to identify and store information about the latest copy of the data blocks. Here the data blocks are checked for consistency. This information is stored in a lookup table referred to as “cache table” or “cache”. This process is called “Cache update or Cache build”. During the cache update, information about previous copy (immediate previous consistent copy of the latest consistent copy) of all the configured blocks is also stored.

Only after the above activities, any read/write/invalidate request can be taken up for execution.

The GC process is triggered if the threshold can be crossed due to the execution of the user requested write/invalidate operation. It may happen that the user requested write shares the wordline with an already written block referred to as “previously written block(s)”. In this case, the page count of the user requested block as well as the “previously written block(s)” is considered for evaluation. This is because, in case of a wordline issue, both the previously written block(s) as well as the current user block needs to be written in the available space.

First, the “previously written block(s)” is checked. If the previously written blocks can be accommodated (block is normal and there is space for this block without crossing the threshold OR block is immediate and there is space for this block within end of the active sector), the user block can start in the free page of the same wordline. The user block is checked now for threshold mark, starting from the free page of the same wordline.

In case the previously written block cannot be accommodated, the user block is started at the next free wordline, skipping the free pages of the current wordline, which has the previously written block. The user block is checked now for the threshold mark, starting from the new wordline.

GC is triggered if the requested user write/invalidate operation cannot be accommodated before the threshold area of the active sector.

6.3.1.1 Support for Unconfigured blocks

The term “unconfigured blocks” refers to the data blocks which are present in DF_EEPROM but are not part of the currently active FEE block configuration set. The FEE driver supports copying of unconfigured blocks to new sector during GC when the user configures FeeUnConfigBlock as FEE_UNCONFIG_BLOCK_KEEP. In this case, FEE driver copies the latest copies of the consistent unconfigured blocks to the new sector.

6.3.2 Robustness Features

As mentioned, the data block content is stored in DF_EEPROM in the form of pages. During programming of the data block, the FEE driver verifies the contents of the entire wordline. This not only ensures that the intended data is written as desired, but it also checks that the other pages of the wordline are NOT affected by the write operation. In order to perform the verification, the FEE driver reads the wordline content (if required) and stores it in RAM before performing the write operation.

In case of a data verification failure, FEE driver internally performs another attempt to write the affected data block(s) (if any) and then the user requested block (all pages), starting from the beginning of the next free wordline. If the data verification fails again while writing the "previously written block(s)", an illegal notification is raised to the user and the FEE driver enters the FAIL state. The user is expected to give a power-on reset followed by reinitialization of the FEE driver.

6.3.3 GC Start Disable/Enable Feature

Usually during post-drive/after-run (upon ECU switch-off request), the application would be required to store the data into data flash and hence write requests would be issued to FEE. If the threshold in the active sector is exceeded during the execution of these requests, then GC will be triggered thus delaying the ECU shutdown or may lead to an aborted GC. In such cases the user can use the GC start disable feature to prevent the start of GC thus providing a graceful shutdown.

Note: Refer sections [6.2.14](#) and [6.2.15](#) for the description and usage of the Disable, Enable GC start functions

6.3.3.1 Disable GC behaviour in case of Immediate Write requests

After Fee_17_DisableGcStart API is called, all requests are executed as usual until threshold is reached. Subsequent immediate writes will NOT TRIGGER GC. However, the immediate write requests will be executed and written in the threshold area until the active sector is full. The next write request (normal/immediate) will be put to pending and will not be serviced until GC is performed.

In order to trigger GC,

- Fee_17_EnableGcStart API needs to be called OR
- In case of a power-on reset followed by FEE driver initialization, the GC-Start is enabled by default. GC will get triggered by the first write/invalidate request.

6.3.3.2 Disable GC behaviour in case of Normal Write / Invalidate requests

After Fee_17_DisableGcStart API is called, all requests are executed as usual until threshold is reached. Subsequent normal write / invalidate request will NOT TRIGGER GC, however, it will be accepted and put to pending.

Note: In case the user wants to write immediate data, the user should cancel the pending request using Fee_Cancel and then issue the write request for the immediate data.

In order to trigger GC,

- Fee_17_EnableGcStart API needs to be called OR
- Incase of a power-on reset followed by FEE driver initialization, the GC-Start is enabled by default. GC will get triggered by the first write/invalidate request.

6.3.4 Behaviour of Write Requests

The write requests (normal/immediate data) will be handled as mentioned in the table below.

Table 71 Behaviour of Write Requests

	NORMAL DATA		IMMEDIATE DATA	
	Threshold NOT reached	Threshold exceeded	Active sector is NOT full	Active sector is full
No Pending request	1. Fee_Write returns E_OK 2. Request is executed without delay 3. State switched to MEMIF_BUSY	1. Fee_Write returns E_OK 2. Request is delayed until "Sector switch" is performed. 3. State switched to MEMIF_BUSY	1. Fee_Write returns E_OK 2. Request is executed immediately (writing into threshold area is allowed) 3.State switched to MEMIF_BUSY	1. Fee_Write returns E_OK 2. Request is delayed until "Sector switch" is performed. 3.State switched to MEMIF_BUSY
Pending request is present	1. Fee_Write returns E_NOT_OK	1. Fee_Write returns E_NOT_OK	1. Fee_Write returns E_NOT_OK	1. Fee_Write returns E_NOT_OK

6.3.5 Behaviour incase Write/Erase failure during GC

For the successful operation of FEE, it is required that the requests to the FLS driver and hence to the data flash are executed successfully, more so during its internal operations (i.e. "InitGC", write of "previously written block"). However, in case of such an eventuality (due to HW errors; data verification failure – more than 1 failure per block), FEE driver reports the error to DEM, if configured and Illegal state notification function is called, if configured, and all the ongoing FEE activities will be stopped and a FAIL state is reached. The user is not supposed to issue any further requests to FEE.

Table 72 Behaviour in various failed GC states

No	Failure in GC State	Pending request	Behaviour
1	All GC states	No Pending request	The write/read/invalidate request is registered. Registered request is not executed Further read/write/invalidate requests will be rejected with E_NOT_OK
2	All GC states	Pending request present	Already registered/pending request is not executed Further read/write/invalidate requests will be rejected with E_NOT_OK

Note: The FEE Driver copies only the consistent blocks during GC and doesnot expect any read errors (Non correctable ECC error) and hence occurrence of error in this case will lead to a FAIL state and the ongoing garbage collection gets aborted at that stage. Rationale: This is done in order to avoid the loss of a consistent block due to a momentary read error during GC. Moreover such a fail state is temporary i.e., FEE will be functional and garbage collection will continue in the subsequent cycle (FEE Initialization).

6.3.6 Illegal State Notification

Illegal state notification is raised under following scenarios:

- During Initialisation of the FEE driver, FEE reads the state pages and the content of the same leads to an invalid / illegal state. Eg: Valid State Page is missing in both of the FEE sectors.
 - The only mechanism for recovering from this Illegal State is by erasing of the DF_EEPROM area. The user can enable FEE to erase both the sectors (not an API request) by configuring FeeEraseAllEnable to TRUE, such that the automatic erase of both sectors is executed. Refer section 4.5.9.
- FEE reaches illegal state if it detects a virgin (all 0x00) flash (DF_EEPROM) when FeeVirginFlashIllegalState is configured as 'true' and FeeEraseAllEnable is configured as 'false'.
- A hardware error occurs resulting in a failure of erase / copy (both write, read) operation during GC.
 - The Erase All behaviour is not applicable for this scenario i.e. the FEE driver will not erase DF_EEPROM, even if FeeEraseAllEnable is configured as TRUE.
- A data verification failure (> 1 failure per block) occurs during GC copy process
- A data verification failure (> 1 failure) occurs for the "previously written block" during user write activity
- Fee_17_IllegalStateNotification function is called by the underlying FLS driver.

In the above cases, the user is recommended to give a power-on reset.

Note: During the first switch on (i.e. in factory), the FEE driver marks the state pages to indicate a valid state. (This operation is indicated by FEE status != MEMIF_IDLE). It is to be ensured that there are no interruptions during this FEE operation else the FEE might reach the Illegal state.

Note: If FEE finds an all 0 DF_EEPROM (both half of the FEE sectors are erased), then FEE does not erase the DF_EEPROM again. The assumption is that the DF_EEPROM is erased correctly. Possible incompletely erased DF_EEPROM, (while programming might) might lead to FEE reaching an illegal state.

6.3.7 Interruption of GC

As mentioned in section [6.3.1](#), the important steps of garbage collection (GC) i.e. End of Copy, End of Erase is marked in the state pages to detect GC state incase of an interruption/reset. On the subsequent start, the state page information is read and hence point of GC interruption, if any, can be found out and then GC can be resumed from this state.

If the copy process was interrupted or not marked, the unfilled sector is first erased and the copy process from the active sector is restarted from the beginning.

If the erase process was interrupted or not marked, an erase activity is reinitiated on this sector. The erased sector is then verified and the sector is marked as ERASED.

6.3.8 Interruption of Write request

When a user write request being executed is interrupted by a reset, the requested data block may not have been completely written. During the subsequent startup (cache build), the interruption of block write is detected by missing marker page (of the data block) and the incompletely programmed block will be ignored and the latest consistent copy of the block is considered.

However in very rare case, if the programming of the first (header) page of the data block is interrupted at a very early stage then the page would be slightly/weakly programmed, this is termed as “early interrupted write”. A slightly/weakly programmed page might be read as an empty page. Early interrupted write is detected during verification of the next write operation using the PVER bit, which ensures the strict margin verification of the programmed data. This is done by using the FLS_17_Pmu_CompareWordsSync API of the FLS driver. In such cases, FEE performs a re-attempt to program the affected data starting from the beginning of the next free wordline. Refer section [6.3.2](#).

6.3.9 Behaviour incase Write/Read failures

If the FEE read operation encounters an uncorrectable ECC error then, FEE_E_READ DEM will be raised by FEE, if configured. The FEE job result is updated to MEMIF_JOB_FAILED and the error notification of the upper layer is called, if configured.

If the FEE Write/Invalidate operation encounters a data write or verification failure (>1 per block), then FEE_E_WRITE / FEE_E_INVALIDATE DEM will be raised by FEE, if

configured. The FEE job result is updated to MEMIF_JOB_FAILED and the error notification of the upper layer is called, if configured.

If the FEE Write/Invalidate operation encounters a data write or verification failure (>1 per block), during the programming of “previously written block“, then FEE_E_WRITE / FEE_E_INVALIDATE DEM will be raised by FEE, if configured. FEE enters a fail state and an Illegal State Notification function of the upper layer is called, if configured.

6.3.10 FEE behavior for GC_RESTART_INIT and GC_RESTART_WRITE configurations

If FeeGcRestart is configured to FEE_GC_RESTART_INIT, the Fee_Init requests Fee_MainFunction to perform FEE Initialization sequence which consists of Init GC and Cache Update. In case FeeGcRestart is configured to FEE_GC_RESTART_WRITE, the first Fee_Read / Fee_Write / Fee_Invalidate requests Fee_MainFunction to start FEE Initialization sequence. In both cases, the Read / Write / Invalidate requests are accepted and kept pending until the cache update is done. Fee_MainFunction executes the pending requests once the cache update is done.

If FeeGcRestart is configured as FEE_GC_RESTART_WRITE then, the module status is set to MEMIF_IDLE at the end of Fee_Init.

If FeeGcRestart is configured as FEE_GC_RESTART_INIT then, the module status is set to MEMIF_BUSY_INTERNAL at the end of Fee_Init. When the initialization is completed by Fee_MainFunction then, the status will be set to MEMIF_IDLE.

The following sequence shows Fee Behaviour in case of FEE_GC_RESTART_INIT configuration.

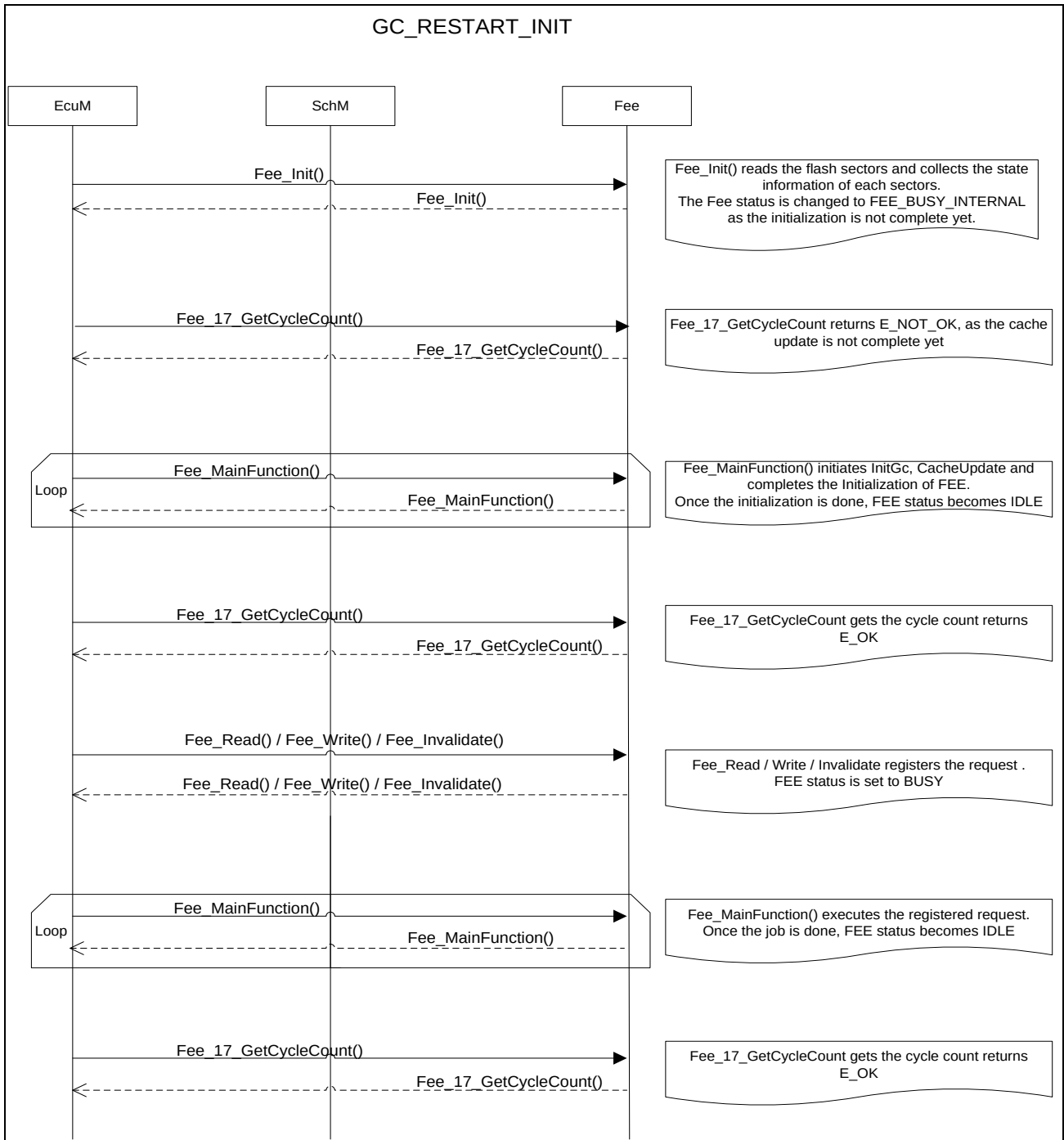


Figure 4 Fee Execution in FEE_GC_RESTART_INIT

The following sequence shows Fee Behaviour in case of FEE_GC_RESTART_WRITE configuration.

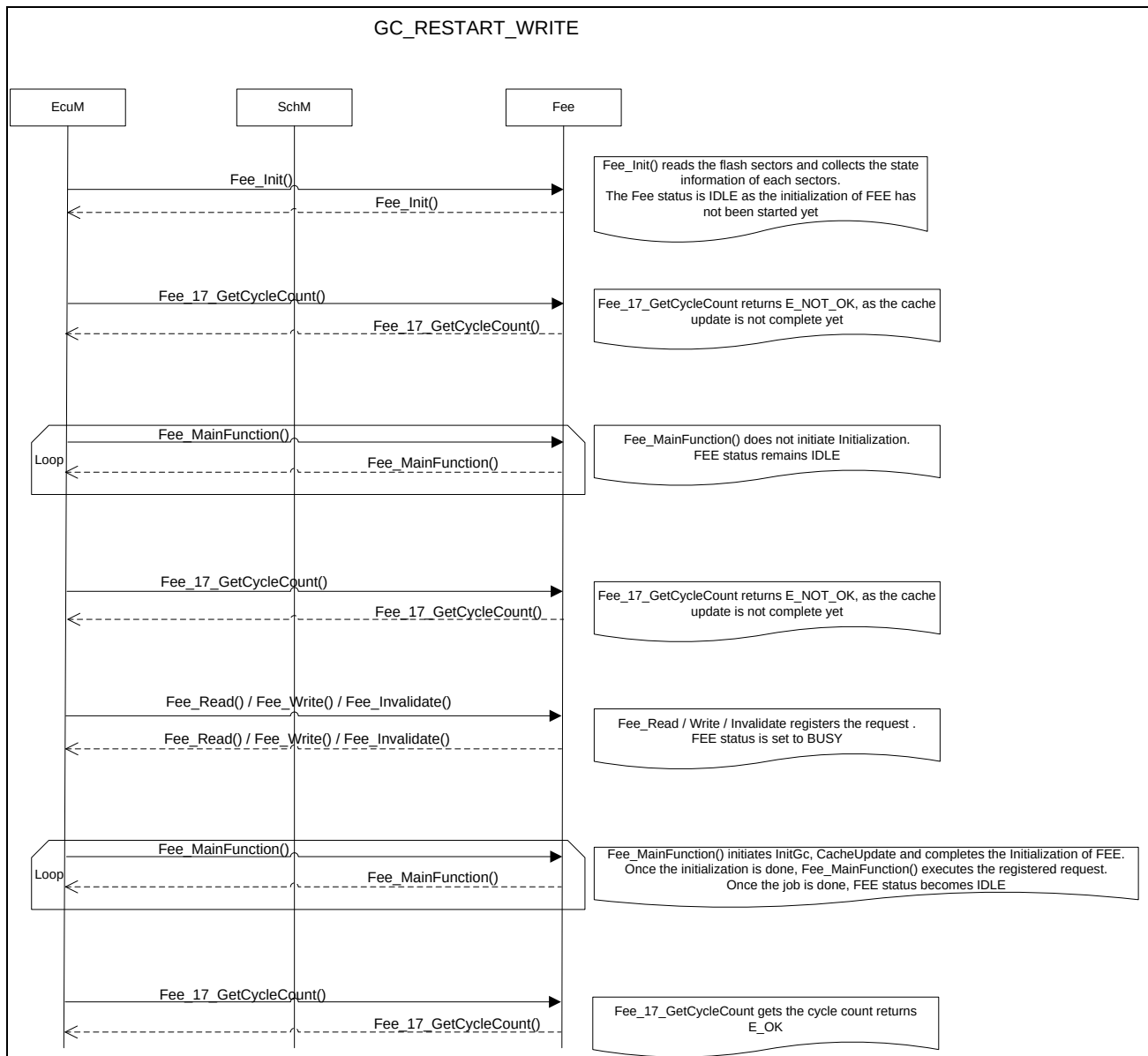


Figure 5 Fee Execution in FEE_GC_RESTART_WRITE

7 Data Description

The below table lists all the global and static variables used (elements of the data structure):

Table 73 Global Data Description

Type/Range	Parameter	Description
Fee_Sector InfoType	FeeSectorInfo[2]	FEE sector information
Fee_Cache Type	FeeBlockInfo[Total number of blocks]	Cache table information. FeeMaxBlockCount + 10. Where 10 is

		the number of safety buffers. This is done to take care of (by only 10) the under-estimated value of FeeMaxBlockCount.
Fee_LastWrittenBlkInfoType	FeeLastWrittenBlkInfo	Last written block information
Fee_GcBlkInfoType	FeeGcCurrBlkInfo	Information of current block being written during GC
Fee_PendReqBufType	FeePendReqInfo	Pending request information
uint32	FeeGcLWBGcSrcAddr[FEE_MAX_NUM_OF_BLKES_IN_WL]	Address of blocks in the current WL during GC. Note: The value of FEE_MAX_NUM_OF_BLKES_IN_WL is arrived by considering the memory size occupied by the smallest possible block i.e., an invalidated block (16 bytes) is considered. Hence $FEE_MAX_NUM_OF_BLKS_IN_WL = \text{WordLine Size} / 16$.
uint32	FeeTempArray[2]	Stores temporary data
uint32	FeeStateCount	State count of the latest state page
uint8	FeeReadWriteBuffer[WL Size]	Read/Write buffer to pass to FLS driver
uint8	FeeGcReadWriteBuffer[WL Size]	Read/Write buffer for usage during GC
uint8	FeeLastWrittenBlkBuffer[WL Size - Page Size]	Buffer to store the last written block data
uint32	FeeGcDestAddr	Destination address for copying data during GC
uint32	FeeGcSrcAddr	Source address for copying data during GC
uint32	FeeNextFreePageAddr	Next free page address for next write
uint32	FeeWriteAffectedAddr	Write affected address
uint32	FeeBlockStartAddr	Block start address
uint32	FeeCurrSectSrcAddr	Current sector source address
uint32	FeeUnErasableWLAddrTemp	Temporarily stores the unerasable word line address
uint8*	FeeUserReadDestPtr	address of the buffer during read job
MemIf_JobResultType	FeeJobResult	job result of the ongoing/finished job
Fls_LengthType	FeeLastWriteSize	Last write size
Fls_LengthType	FeeLastReadSize	Last read size

Fee_PageType	FeeBlkPageCount	number of DFLASH pages read/written
uint16	FeeUserWriteBytesCount	Number of bytes written
uint16	FeeCurrReqBlockNum	Block number of the current request
uint16	FeeIntrCurrReqPageCount	Internal current request's page count
uint16	FeeGCCopyIndex	Index of logical blocks covered during GC
uint16	FeeGCUnconfigBlkCopyIndex	Index of un-configured blocks covered during GC
uint16	FeeUnConfigBlockCount	Number of unconfigured blocks to be handled
uint16	FeeGcPrevBlockNumber	Previous block number during GC
uint16	FeeGcFirstBlkNumInWL	Block number of the first block in the WL during GC
Fee_StatusFlagsType	FeeStatusFlags	FEE status flags
uint8	FeeLastWrittenBlockDirty	Last written block's dirty status
uint8	FeePendReqStatus	Pending request status
uint8	FeeGcState	GC state variable
uint8	FeeGcResumeState	State at which GC was interrupted
uint8	FeeGcBlkIndexInWL	Block index in WL during GC
uint8	FeeInitGCState	Current state for FEE initialization
uint8	FeePrepDFLASHState	Current state of DFlash preparation
uint8	FeeCacheState	Current state of cache update
uint8	FeeRepairStep	State page repair status
uint8	FeeWLAffectedType	WL affected type
uint8	FeeIntrJob	Current internal job
uint8	FeeIntrJobStatus	Current internal job status
uint8	FeeUserJobStatus	Current user job status
uint8	FeeIntrJobResult	Current internal job result
uint8	FeeUserJobResult	Current user job result
uint8	FeeMainJob	Current main job
uint8	FeeUserJobFailCount	User job fail count
uint8	FeeIntrJobFailCount	Internal job fail count
uint8	FeeUncfgBlksExceeded	Stores the status whether the number of un-configured blocks exceeded the configured limit
uint8	FeeUnErasableWLCntTemp	Temporarily stores unerased word line count
uint8	FeeEraseSuspendStatus	Status of Erase suspend: [0 = Erase not suspended; 1 = Erase suspended;

CONFIDENTIAL

Data Description

		>1 = Erase Resume failed N number of times]. Where N = (FeeEraseSuspendStatus - 1).
uint8	FeeEraseResumeDemReported	Status of Erase-Resume failure DEM report
boolean	FeeDisableGCStart	GC disable/enable status

8 Error Classification

The Driver provides pre-compile switches, so that the production errors are reported to DEM module only if it is enabled.

Table 74 Error Classification

Error Code	Error	Relevance	Value (hex)
FEE_E_UNINIT	API Service called without Fee_Init being called	Development	0x01
FEE_E_INVALID_BLOCK_NO	API service called with wrong Block number	Development	0x02
FEE_E_INVALID_BLOCK_OFS	API service called with wrong Block Offset	Development	0x03
FEE_E_INVALID_DATA_PTR	API service called with invalid pointer	Development	0x04
FEE_E_INVALID_BLOCK_LEN	API service called with wrong Block Length	Development	0x05
FEE_E_BUSY	API service called when FEE is BUSY with a previous request	Development	0x06
FEE_E_BUSY_INTERNAL	Fee_SetMode API is called when FEE status is MEMIF_BUSY_INTERNAL	Development	0x07
FEE_E_INVALID_CANCEL	Fee_Cancel API is called when FEE status is not MEMIF_BUSY (i.e., no user requested job is on-going)	Development	0x08
FEE_E_GC_INIT	Error (SQER, OPER, PROER) occurred during InitGC activity	Production	Configurable by user
FEE_E_WRITE	Error (Non correctable ECC error, SQER, OPER, PROER) occurred when executing the user write request	Production	Configurable by user
FEE_E_READ	Non correctable ECC error detected when executing the user read request (old data, latest data)	Production	Configurable by user
FEE_E_GC_READ	Error (Non correctable ECC error) occurred when performing a Read operation in Garbage Collection phase	Production	Configurable by user
FEE_E_GC_WRITE	Error (SQER, OPER, PROER) occurred when performing a Write	Production	Configurable

	operation in Garbage Collection phase		by user
FEE_E_GC_ERASE	Error (SQER,OPER, PROER) occurred when performing/resuming an Erase operation in Garbage Collection phase	Production	Configurable by user
FEE_E_INVALIDATE	Error (Non correctable ECC error, SQER,OPER, PROER) occurred while executing user requested block invalidate operation	Production	Configurable by user
FEE_E_WRITE_CYCLES_EXHAUSTED	Error because configured limit of write cycles is exhausted for the given block	Production	Configurable by user
FEE_E_GC_TRIG	Error because of insufficient space in the new sector for data block copy / state page programming during GC	Production	Configurable by user
FEE_E_UNCONFIG_BLK_EXCEEDED	Error due to shortage of space in cache table to accommodate the detected un-configured block.	Production	Configurable by user

9 Example Usage

This chapter describes how the FEE driver can be configured and how to use different API provided by Fee driver.

10 Configuring FEE Driver

The configuration of the FEE driver shall be done as per the description provided in [Section 4](#).

Configuration dependency:

1. Configuration of FLS module
2. Configuration of system clock shall be done in MCU module

10.1 Using FEE services

10.1.1 Function call example

Code Listing 1 Function call example

```
001:  /* Fls_17_Pmu_MainFunction and Fee_MainFunction shall be called
      cyclically throughout */
002:
003:  /* Fee_Init must be called before performing any FEE job */
004:  Fee_Init();
005:  while(Fee_GetStatus() != MEMIF_IDLE) ;
006:
007:  /* Write block 1 */
008:  ret_fee = Fee_Write(1,my_fee_buffer2);
009:  while(Fee_GetStatus() != MEMIF_IDLE) ;
010:
011:  /* Write block 2 */
012:  ret_fee = Fee_Write(2,my_fee_buffer1);
013:  while(Fee_GetStatus() != MEMIF_IDLE) ;
014:
015:  /* reading first 10 bytes of Block 1 */
016:  ret_fee = Fee_Read(1,0,my_read_buffer,10);
017:  while(Fee_GetStatus() != MEMIF_IDLE) ;
018:
019:  /* Write block 1 */
020:  ret_fee = Fee_Write(1,my_fee_buffer4);
021:  while(Fee_GetStatus() != MEMIF_IDLE) ;
022:
023:  /* reading the updated first 10 bytes of Block 1 */
024:  ret_fee = Fee_Read(1,0,my_read_buffer,10);
025:  while(Fee_GetStatus() != MEMIF_IDLE) ;
026:
027:  /* Write block 2 */
028:  ret_fee = Fee_Write(2,my_fee_buffer1);
029:  while(Fee_GetStatus() != MEMIF_IDLE) ;
```

```

030:
031:  /* Read the updated 70 bytes starting from the 10th byte till 80th byte of
    Block 2 */
032:  ret_fee = Fee_Read(2,10,my_read_buffer,80);
033:  while(Fee_GetStatus() != MEMIF_IDLE) ;
034:
035:  job_result = Fee_GetjobResult();
036:
037:  /* Invalidate Block 2 */
038:  ret_fee = Fee_InvalidateBlock(2);
039:  while(Fee_GetStatus() != MEMIF_IDLE) ;
040:
041:  /* Read 70 bytes starting from the 10th byte till 80th byte of Block 2 */
042:  ret_fee = Fee_Read(2,10,my_read_buffer,80);
043:  job_result = Fee_GetjobResult();

```

Note:

The Fee_MainFunction and Fls_17_Pmu_MainFunction should be called periodically by the scheduler

In the above code snippet, while(Fee_GetStatus() != MEMIF_IDLE); is used only to indicate that, before initiating further FEE operations, FEE status needs to be polled for MEMIF_IDLE.

10.1.2 Jump table functionality implementation hint

A software (referred as application software) accesses the FEE APIs and its dependent FLS APIs present in an other software (referred as boot software) via function pointer table (referred as jump table).

Update the boot software's make files such that the files Fee_Fcpt.c and Fee_Fcpt.h are part of the software. Fee_Fcpt.c file contains the jump table. Fee_Fcpt.h contains its dependent type definitions.

Configure FEE driver as per the need of the application software.

Update make files such that FEE configuration files Fee_PBCfg.c, Fee_Cfg.h and Fee.h are part of the application software.

Initialize Fee_CfgPtr with the address of application software's FEE config root.

Initialize Fls_ConfigPtr with the address of boot software's FLS config root.

Fee_CfgPtr and Fls_ConfigPtr should be located in the same address as where they were located in the boot software.

Fee_Header_Application_Const should be initialized with the address where Fee_Header_Const is located in the boot software.

Example code for application software:

```

#define Fee_Write ((Fee_Write_Type)Fee_Header_Application_Const[0])
#define Fee_Read ((Fee_Read_Type)Fee_Header_Application_Const[1])
#define Fee_InvalidateBlock ((Fee_InvalidateBlock_Type)Fee_Header_Application_Const[2])

```

```
#define Fee_Cancel ((Fee_Cancel_Type)Fee_Header_Application_Const[3])
#define Fee_GetStatus ((Fee_GetStatus_Type)Fee_Header_Application_Const[4])
#define Fee_GetJobResult ((Fee_GetJobResult_Type)Fee_Header_Application_Const[5])
#define Fee_EraseImmediateBlock ((Fee_EraseImmediateBlock_Type)Fee_Header_Application_Const[6])
#define Fee_Init ((Fee_Init_Type)Fee_Header_Application_Const[7])
#define Fls_17_Pmu_Init ((Fls_17_Pmu_Init_Type)Fee_Header_Application_Const[8])
#define Fls_17_Pmu_MainFunction ((Fls_17_Pmu_MainFunction_Type)Fee_Header_Application_Const[9])
#define Fee_MainFunction ((Fee_MainFunction_Type)Fee_Header_Application_Const[10])
#define Fee_17_EnableGcStart ((Fee_17_EnableGcStart_Type)Fee_Header_Application_Const[11])
#define Fee_17_DisableGcStart ((Fee_17_DisableGcStart_Type)Fee_Header_Application_Const[12])
#define Fee_SetMode ((Fee_SetMode_Type)Fee_Header_Application_Const[13])
#define Fee_17_GetCycleCount ((Fee_GetCycleCount_Type)Fee_Header_Application_Const[14])
#define Fee_17_GetPrevData ((Fee_17_GetPrevData_Type)Fee_Header_Application_Const[15])
```

From the application software call the required FEE and its dependent FLS APIs.

Limitation / use-case: The dependent module (i.e, DET, DEM, SchM) APIs used by FLS and FEE are part of the software in which FLS & FEE drivers exist.

10.1.3 Important Notes about the usage of FEE driver

1. FLS driver should be initialized prior to using FEE driver.
2. Fee_Init must be called prior to calling any FEE job.
3. 'Fls_17_Pmu_MainFunction' and 'Fee_MainFunction' (collectively referred to as 'MainFunction') must be called at regular intervals.
4. The configuration parameter FeeMaxBlockCount must be configured appropriately as mentioned in the description of this parameter. This parameter decides the size of the cache table.
5. A summary of the return values for a Read/Write operation is given below
The return values will mean the following:
 - E_OK – Job is registered. It will be analyzed and executed with a minimum possible delay. Job result at this point will be MEMIF_JOB_PENDING. Later, when the job is actually analyzed and executed, the job result will be changed accordingly i.e. MEMIF_JOB_OK or MEMIF_JOB_FAILED. The user must use this job result and ignore E_OK returned by the function earlier.
 - E_NOT_OK – Job is rejected.
6. When not using the notification functions (pointer to job end and job error notification functions are NULL_PTR), care should be taken for waiting for the FEE driver to come to MEMIF_IDLE state before executing the next FEE job. While notifications provide an easier way to perform jobs without hogging the CPU in the waiting loop. The next FEE job can be executed in the next call of the Fee_MainFunction.

About [Code Listing 1:](#)

1. Line #23 shows an example of reading the updated contents of a logical block.
2. In Line #35, the job_result will be set to MEMIF_JOB_OK.
3. Line #37 shows how to invalidate a particular logical block.
4. Line #42 will attempt to read an Invalidated block. Subsequently, in line #43 the job_result will be set to MEMIF_BLOCK_INVALID.

11 Assumptions and Limitations

11.1 Assumptions and Deviations from the Software Specification

Assumptions and Design Aspects of FEE:

- 1) FEE may be busy with internal operations (denoted by MEMIF_BUSY_INTERNAL) even if user has not requested any operation. However, any request will still be accepted by the FEE driver and will be executed with delay.
- 2) FLS driver should be used only via FEE (as per AUTOSAR specification).
- 3) A return value of E_OK from FEE APIs (read/write) doesn't mean that the job will successfully execute. Instead it just means that the job is registered to be analyzed and executed later. To check the job result, the user must call Fee_GetJobResult API.
- 4) Though FEE algorithm takes care of interrupted GC, the interruption of write operation can lead to erroneous data or reduced data retention for the written and interrupted data page. Therefore any interruption of write operation has to be avoided.
- 5) User must NOT issue multiple requests simultaneously. Instead, the user should take care of issuing a fresh request by performing any of the following actions:
 - a. Poll the FEE driver status using Fee_GetStatus API. Wait until FEE state changes to MEMIF_IDLE or MEMIF_BUSY_INTERNAL.
 - b. Cancel the previous user request (Call Fee_Cancel). Note: Fee_Cancel will cancel only the write/invalidate requests that are pending but any ongoing write/invalidate request is not cancelled. However, read jobs (both pending and ongoing) are cancelled.
- 6) Fee APIs must NOT be pre-empted by other FEE APIs directly/indirectly in order to ensure that there is no data corruption.
- 7) The FEE module's environment shall not call the function Fee_Init during a running operation of the FEE module.

11.2 Limitations

1. Due to the usage of stack based (double-sector) algorithm for FEE, there is a need to maintain a cache table in RAM. The cache table holds the address and status information of the latest copy of logical blocks. The locations of the logical blocks are updated by reading the DF_EEPROM area. Hence the very first read/write/invalidate request might have to wait until the cache table is built / updated. The worst case scenario would be of reading the entire DF_EEPROM area.
2. Interrupted programming of the last page of a logical block (weakly programmed data block marker page – a non-zero value or un-correctable ECC error) will be determined as a consistent block. But during the next initialization (cache table build), if that

weakly programmed page's content is read as 0's (without un-correctable ECC error – i.e., it looks like a missing data block marker page) then the block will be considered as inconsistent.

3. If FEE finds an all 0 DF_EEPROM (both half of the FEE sectors are erased), then FEE does not erase the DF_EEPROM again. The assumption is that the DF_EEPROM is erased correctly. Possible incompletely erased DF_EEPROM, might later (while programming can) lead to FEE reaching an illegal state. This condition is not handled by the FEE driver.
4. (Applicable only for TC29x) For optimal implementation of FEE and effective usage of Fee_MainFunction calls by upper layer, in TC29x it has been chosen to erase (1/2 DF_EEPROM) 384Kbyte with one erase command and as a consequence, suspend times and application reset times are higher according to the Data Sheet / Target Specification.
5. When virgin (all 0x00) DF_EEPROM is detected, FEE formats the DF_EEPROM area (if FeeVirginFlashIllegalState is configured as false) – i.e., it programs Erased and Valid State Page in FEE Sector 0 followed by Erased State Page in FEE Sector 1. This procedure should be uninterrupted. If formatting process gets interrupted before the Valid State Page is programmed, then FEE reaches illegal state and calls the configured illegal state notification function.

Note: State page evaluation relies on state page format and ECC error correction/detection capabilities. Interruption of DFLASH erase process (e.g. ECU power loss) should be prevented, to minimize risk for ECC error detection/correction failures. Although the garbage collection and Flash ECC error detection/correction is robust, there may exist a very rare scenario in which Flash ECC error detection/correction may fail while reading a sector in which Flash Erase operation was aborted, resulting in FEE incorrectly detecting the active sector.

11.3 Deviations

1. Clarification of FEE021 for read requests (Fee_Read / Fee_17_GetPrevData): Due to the usage of the double-sector algorithm, address of the logical block will not be calculated to locate the block in memory, instead a look-up table (cache-table) will be used.
2. Clarification of FEE023: A read request will fail with MEMIF_BLOCK_INCONSISTENT in the following cases:
 - The logical block does not exist.
 - The most recent version of the block is not complete (assuming block number was identified without read error).
 - At least one Flash read access resulted in an error (e.g. when an uncorrectable ECC error was detected).
3. Clarification of FEE037: A block is invalidated by writing appropriate infrastructure data of the block to DF_EEPROM.
4. Clarification of FEE024 and FEE036: Due to the usage of the double-sector algorithm, address of the logical block is not calculated. Instead the block will be written to DF_EEPROM as per the double-sector algorithm.

5. Extension of AUTOSAR SWS for Fee_Write and Fee_InvalidateBlock: If the Block Cycle Counter of the addressed block has already reached its maximum value, the following steps will be executed:
- FEE_E_WRITE_CYCLES_EXHAUSTED error is reported to DEM, if configured.
 - Job result is set to MEMIF_JOB_FAILED.
 - Job Error Notification function will be called, if configured.
6. Violation of FEE080: In case of an ongoing read operation requested by the user, an IFX FEE specific API Fls_17_Pmu_CancelNonEraseJobs will be called, which cancels only an ongoing read job; no development errors are supported by Fls_17_Pmu_CancelNonEraseJobs.
7. Violation of FEE009, FEE080 and FEE081: An ongoing write operation requested by the user will not be affected by Fee_Cancel. Blocks containing immediate data cannot be written instantaneously (in conflict with FEE009). Fls_17_Pmu_Cancel will not be called (in conflict with FEE080). Fee_Cancel will not reset the FEE Module's internal variables (in conflict with FEE081).
8. Violation of FEE009 and FEE080: Ongoing internal read, write or erase operations will not be affected by Fee_Cancel; Fls_17_Pmu_Cancel will not be called (in conflict with FEE080). Blocks containing immediate data cannot be written instantaneously, and the write request is delayed by currently running module internal management operations (in conflict with FEE009).
9. Extension of AUTOSAR SWS for Fee_Cancel: In case that a user request is pending but is not yet being executed, the request can be canceled; Fee_Cancel shall only reset the FEE Module's internal variables. During garbage collection, the status shall switch back from MEMIF_BUSY to MEMIF_BUSY_INTERNAL when the garbage collection is still running or to MEMIF_IDLE when it has finished.
10. Extension of FEE090: Fee_GetStatus will return MEMIF_BUSY not only while a job is executed but also while it is waiting for execution. During garbage collection, the status MEMIF_BUSY_INTERNAL will be returned only before a new request is accepted; then the status will switch to MEMIF_BUSY.
11. Clarification of FEE067: Since double-sector algorithm is used with threshold limit for triggering GC, immediate block's write requests during GC can be accommodated within the pre-erased threshold area of the active FEE sector. Hence, the API Fee_EraseImmediateBlock is implemented as a dummy function.
12. Fee_Cancel API shall report the error FEE_E_INVALID_CANCEL to DET, if there is no job (no ongoing user requested job) to be cancelled. Refer http://www.autosar.org/bugzilla/show_bug.cgi?id=43545. This is inline with AS4.0.3 requirements.
13. The development error FEE_E_BUSY_INTERNAL is not supported (except for Fee_SetMode API), as the user request will be accepted by the FEE driver (but will be executed with delay) even when the FEE driver status is MEMIF_BUSY_INTERNAL.
14. To allow FEE to be shared for multiple locations, some parameters have been modified as Post Build parameters. Refer section 4.
15. Few of the configuration parameters are not user configurable (fixed). Refer section 4.

11.4 Hardware features that are not supported

None

www.infineon.com